UNIVERSITÉ LIBRE DE BRUXELLES

Faculté des Sciences

Département d'Informatique

An Experimental Framework for Evaluating Traffic Deviation Plans

Callens Hugo

Promotors :
Prof. Gianluca Bontempi
Eladio Montero Porras
Davide Andrea Guastella

Master Thesis in Computer Sciences

Academic year 2025 – 2026

Contents

1 Introduction

1.1 Digital Twin

A **digital twin** is a data-driven virtual representation of real-world entities designed to mirror a physical object or system throughout its lifecycle. The virtual part is updated with real-time data gotten from sensors or other sources making it possible to link the physical and digital parts. [**defDT**, **IBMDT**]

A digital twin consists of three components [**DT_health**] :

1. **Physical entity** : The real-world object or system being modeled (e.g., traffic, a plant)
2. **Virtual representation** : A model that replicates the behavior of the physical entity.
3. **Connection** : The link between the physical system and the virtual system making it possible to update the physical system by exchanging information via this real-time connection.

Figure 1 summarises this relationship between the physical object, the virtual model and the data connection.

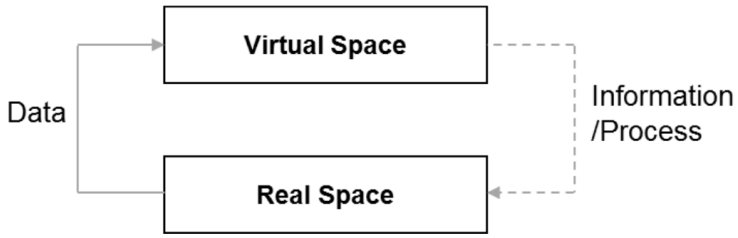


Figure 1: Illustration of a digital twin from [**digital_twin**]

Digital twins receive continuously streams of data from sensors or other sources ensuring that the virtual model replicates the current state of the physical object. The main goals for a digital twin are to simulate the physical entity and predict what could happen in order to optimize or predict failures. Urban traffic networks, which are both data-rich and fast-evolving, provide fertile ground for the deployment of digital twins. Figure 2 gives an example workflow for linking SUMO to map and sensor inputs in this context. Digital twins in traffic can potentially simulate and optimize traffic flows, supporting smarter city planning and real-time operations. A detailed discussion of modern digital twin solutions in traffic management is provided in **Section 2.1.1**.

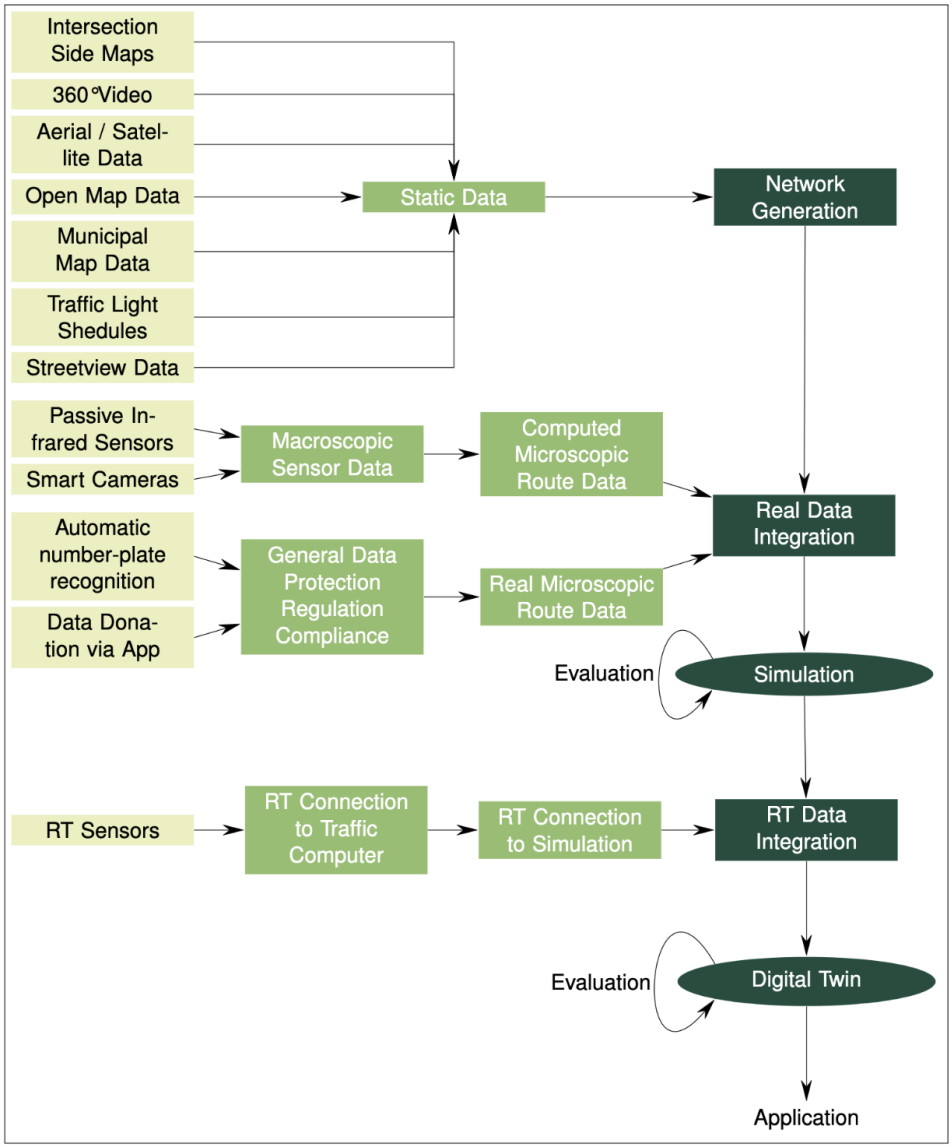


Figure 2: Example workflow for driving SUMO towards a digital twin. Various static and real-time data sources (maps, sensors, cameras) are integrated into simulation models to continuously update and calibrate the digital twin for real-world application. Source : [**sumo_twin**]

1.2 Context

In this project, a digital twin acts as a precise and efficient method to simulate traffic based on real-world data. Digital twins leverage continuous data streams (for example, from vehicle counting devices) to create an up-to-date virtual model of physical traffic flows. Accurate traffic simulation relies on high-resolution data, typically gathered by a variety of counting devices such as inductive loop traffic detectors, Bluetooth recognition systems and ANPR (Automatic Number Plate Recognition) cameras [**counting_devices**]. These devices measure the number of vehicles passing specific locations over time, producing the data needed for dynamic modeling and simulation.

Through the aggregation and analysis of such data, it becomes possible to estimate traffic

density and infer vehicle movement patterns across a road network. In this framework, physical traffic corresponds to the physical entity of the digital twin, real-time vehicle counts represent the virtual layer and the counting devices themselves serve as the communication link between the physical and digital components.

Traffic simulation is particularly vital in urban environments such as Brussels, where the frequent congestion and ongoing roadworks demand dynamic management of the transport network. Planned deviations due to construction may alter typical traffic flows and produce unexpected bottlenecks. Accurate simulation enables city planners to evaluate alternative deviation plans and identify those that minimize congestion and disruption. Additionally, the direct and indirect effects of traffic interventions, such as road closures, can be promptly assessed.

1.3 Objective

The main objective of this work is to develop a reproducible experimental framework for evaluating deviation plans when a road segment is closed. Instead of focusing on the migration of an existing interface, this thesis studies how a SUMO-based traffic digital twin, stochastic routing and repeated simulation can be combined to compare candidate detours in a structured way. The work is divided into several sub-goals.

- **Scenario generation:** The construction of a synthetic, network-aware demand scenario from a SUMO road network and Traffic Analysis Zone data.
- **Deviation-plan computation:** The generation of candidate detour paths around a closed edge or sequence of edges, together with route-rewriting rules that apply these plans to affected vehicles.
- **Stochastic simulation:** The execution of repeated SUMO simulations using randomized routing seeds and paired baseline runs in order to capture variability in route choice and traffic outcomes.
- **Evaluation and decision support:** The comparison of candidate plans using traffic indicators, paired statistics, feasibility checks and multi-criteria decision analysis.
- **Framework validation:** The application of the full pipeline to controlled experiments and a Brussels neighbourhood case study in order to assess its usefulness and limitations.

Ultimately, the goal is to support city planners and traffic managers with a data-driven method for comparing deviation plans before they are deployed on the road network.

2 Foundations and Related Work

2.1 Traffic digital twins for decision support

2.1.1 Digital twins in traffic simulation

A traffic digital twin is useful here less as a real-time control system than as a planning environment in which a road manager can ask a concrete counterfactual question: *what happens if a road segment is closed and traffic is assigned to one of several deviation plans?* The literature on digital twins for mobility emphasizes this ability to reproduce a road network, ingest transport data, simulate alternative scenarios and support decision making before actions are deployed in the physical network [**data_mining_dt**, **intelligent_transport**, **digital_twin_transport**]. This thesis uses that same decision-support idea, but it implements it as an offline experimental pipeline rather than as a continuously synchronized operational twin.

The framework developed in this thesis therefore separates the physical city from the experimental model used for plan comparison. The physical layer is represented by a SUMO road network and traffic analysis zones, the scenario layer generates synthetic, network-aware origin-destination demand, the planning layer converts a selected closure into candidate detour paths and the evaluation layer simulates and ranks those candidates. In this sense, the framework should be understood as an offline decision-support tool rather than as a fully operational traffic-management digital twin. It does not continuously ingest live sensor data or maintain a real-time estimate of the current traffic state. Its contribution lies instead in enabling the reproducible comparison of deviation policies under controlled assumptions.

The result is a decision-support digital twin prototype for roadwork planning. Its purpose is not to predict the exact state of Brussels traffic at a particular minute. Instead, it provides a repeatable environment for testing how different detour definitions change route assignment, vehicle time loss, travel time and the number of affected users. This framing is important because roadwork decisions are often made before the disruption occurs. The planner needs evidence about candidate plans, not only a live dashboard after congestion has already formed.

This scope follows from the exploratory work carried out before the present implementation. Several digital-twin approaches were considered: broad smart-city twins that integrate many urban systems, operational traffic twins that continuously assimilate sensor data, SUMO-based twins focused on traffic simulation and *TrafficTwin*, an earlier interactive prototype for deviation-plan assessment [**smart_city**, **digital_twin_transport**, **sumo_twin**, **SIM_param**]. TrafficTwin is the closest predecessor because it already frames a closure as a planning problem and lets a user inspect the impact of a deviation plan. The difference is that this thesis turns that idea into a reproducible experimental pipeline: demand generation, candidate-plan generation, stochastic re-routing, simulation, statistical comparison and multi-criteria ranking are explicit stages rather than manual or single-run interactions.

2.1.2 Simulation and mobility

Traffic simulation is the mechanism that turns the digital-twin idea into an experiment. The common theme across simulation-based traffic studies is that a proposed intervention can be tested in a controlled environment before it is implemented on the road. Prior work uses this principle for adaptive signal control, co-simulation and AI-based traffic management [**ai_driven_simulation**, **cosimulation**, **traffic_light_simulation**]. Those studies are useful background, but they usually use simulation to design or assess systems that adapt during operation, for example a controller that changes signal phases or an AI method that learns traffic-management actions. The intervention studied here is different. The closure and candidate detour policies are defined before the simulation runs and simulation is used to measure how each planned response affects delay, travel time and impacted users.

Another related line of work concerns real-time route recommendations from navigation applications such as Google Maps[1] and Waze[2]. Scientific studies of online routing use these services as examples of systems that can redirect demand dynamically and, if many users react to similar information, can also create network-level effects that differ from a system optimum [**jalota_online_routing_2021**, **toso_route_recommendations_2023**]. This literature is relevant because it treats routing guidance as an intervention on traffic flow. The present thesis differs from it in both time scale and control objective: it does not provide live individualized advice to drivers, but evaluates planned deviation policies offline before a roadwork is deployed.

The simulation problem addressed here has three practical requirements. First, the same demand must be reused across alternatives so that candidate plans are compared on equal terms. Second, the experiment should test the sensitivity of plan rankings to plausible route-choice dispersion. Traffic assignment theory distinguishes deterministic shortest-path or user-equilibrium assignment from stochastic formulations in which travellers choose between alternatives according to perceived generalized costs rather than a single perfectly observed cost [**modelling_transport**]. Third, the final recommendation must combine several outputs, since a plan that reduces one metric may worsen another. For example, a detour can reduce queueing and time loss near the closed edge by spreading traffic over alternative roads, while also increasing the mean travel time of affected vehicles because they are sent through a longer path.

In this setting, simulation is not used only for visualization. It is the measurement instrument of the thesis. The output of each SUMO run is parsed into quantitative indicators and the evaluator uses the distribution of those indicators over repeated runs to decide which deviation plan is most robust for the selected closure.

2.1.3 Positioning of this work

The problem addressed in this thesis is narrower than building a complete platform for traffic management. The practical question is: *if a road edge is closed, how can several candidate deviation plans be compared before the closure is deployed?*. Existing work provides useful

[1] https://www.google.com/maps/

[2] https://www.waze.com/

pieces of that answer. SUMO studies explain how to build reproducible simulation scenarios from a road network and demand data [**sumo_tutorial**, **sparse_data_SUMO**]. Transport digital-twin research motivates the use of simulation models to test planning decisions before acting on the physical network [**intelligent_transport**, **digital_twin_transport**]. Online routing studies show that routing guidance can change network flows, although they usually study live advice to individual drivers rather than planned roadwork policies [**jalota_online_routing_2021**, **toso_route_recommendations_2023**]. At the tool level, *TrafficTwin* is the closest predecessor because it already treats a road closure as a deviation-plan assessment problem [**SIM_param**].

The contribution of this thesis is to connect these ideas into one reproducible evaluation pipeline. More specifically, it generates synthetic demand for a SUMO network, constructs candidate detours around a selected closure, rewrites affected routes, reruns the same scenario under stochastic routing assumptions and ranks the resulting plans with paired statistics and multi-criteria decision analysis. The emphasis is therefore not on a new simulator or a real-time routing service, but on a transparent workflow for comparing roadwork deviation policies under the same experimental conditions.

2.2 Traffic simulation tools

2.2.1 Purpose of traffic simulation tools

Traffic congestion is one of the major challenges faced by modern cities and is expected to get worse with increasing population and vehicle usage [**increasing_traffic**, **increasing_cars**]. Traffic simulation tools such as SUMO[3], Vissim[4], Aimsun[5], or CityFlow[6] are mathematical and computational models used to represent traffic dynamics, test network designs and estimate operational or environmental effects before an intervention is implemented [**modelling_transport**, **SUMO_sumo**].

2.2.2 Levels of detail in traffic simulation

Traffic simulations are commonly categorized into three main types based on the level of detail desired [**modelling_transport**, **SUMO_sumo**]:

1. **Microscopic Simulation**: Models each traffic participant (e.g., vehicles, pedestrians) individually. Agents continuously in time and space respond to their immediate environment. The global traffic dynamics is the result of the sum of these individual decisions and behaviors.

2. **Mesoscopic Simulation**: Still represents individual agents, but their behavior is determined by average traffic conditions like speed or density. Agents only interact directly at key points in the network, such as intersections.

3. **Macroscopic Simulation**: Looks at traffic as a whole rather than focusing on

[3]https://sumo.dlr.de/
[4]https://www.ptvgroup.com/en/products/ptv-vissim/
[5]https://www.aimsun.com/
[6]https://cityflow-project.github.io/cityflow/

individual vehicles.

Choosing a simulation type entails balancing computational cost and realism. For instance, microscopic simulation is ideal for projects that require modeling of individual behaviors or the testing of interventions at the street or intersection level, which is often necessary for a dense urban environment such as Brussels. In contrast, a macroscopic simulation might suffice for citywide traffic forecasting or strategic planning, where the focus is on broader patterns rather than local details.

2.2.3 SUMO as the selected microscopic simulator

SUMO (Simulation of Urban MObility) is an open-source, highly portable, microscopic and continuous mutli-modal traffic simulation tool. It allows detailed modeling of individual vehicles, each following its own route through a specified road network. As a microscopic simulator, SUMO treats every vehicle as an individual entity with specific behaviors and interactions. The tool is multi-modal enabling the simulation of diverse traffic participants including cars, public transport vehicles and pedestrians [**SUMO_eclipse**] [**SUMO_homepage**] [**SUMO_doc_glance**].

SUMO has been used extensively to model real-world traffic systems in diverse urban contexts, including cities such as Berlin, Dublin and Turin [**SUMO_scenarios**]. Each scenario adapts to the specific traffic dynamics and urban characteristics of its location :

1. **Berlin:** Berlin has served for large-scale traffic simulations using SUMO. The "24 Hours of Traffic in Berlin" scenario captures over 2.2 million trips within an $800km^2$ area. This model enhances traditional traffic simulations by incorporating motorized private vehicles alongside realistic bus schedules derived from actual General Transit Feed Specification (GTFS) data, representing 89% of the city's bus lines. This scenario allows for holistic analyses that combine traffic, communication (including V2X and LTE/5G) and application simulations, making it suited for smart mobility researches. [**berlin_scenario**]

2. **Dublin:** The study by Gueriau and Dusparic [**dublin_scenario**] leverages SUMO to simulate and analyze the impact of connected and autonomous vehicles (CAVs) on real Irish road networks, covering urban, national and motorway settings. Using detailed demand and network data from Dublin and surrounding areas, the research quantifies how varying CAV penetration rates affect both traffic efficiency and open dataset for benchmarking the effects of emerging vehicle technologies in mixed traffic environments.

3. **Turin:** Leveraging real-world data, the TuSTScenario [**turin_scenario**] provides a highly detailed SUMO-based simulation of metropolitan Turin, covering over $602.61km^2$. The model integrates structural road information from OpenStreetMap, validated average speeds from Google APIs, actual traffic light programming and real trip demands from local mobility services. With more than 2.2 million simulated daily trips and validated against city sensor data, this scenario enables realistic studies of urban traffic dynamics and congestion patterns.

Brussels is also a large city warranting similarly large-scale simulations. However, the focus of this study is distinct: rather than prioritizing multi-modal integration or CAV

impact, we specifically investigate the planning and management of roadworks within the Brussels network. The goal is to demonstrate how such interventions can be realistically modeled, analyzed and optimized in SUMO for modern urban infrastructure management.

SUMO has been used to evaluate the effects of technologies like virtual traffic lights communication systems where cars directly communicate with each other. [**SUMO_VTL**] In this study, Tonguz found that virtual traffic lights could reduce congestion and the likelihood of traffic accidents. Moreover SUMO can be used in environmental research, for instance in assessing the impact of traffic-related emissions on urban air quality. In such cases traffic flows are generated from sensor data to estimate real-world emission patterns [**SUMO_emission**].

SUMO runs can be made reproducible when the same version, input files, command-line options and random seed are used. This is useful for controlled experiments because differences between scenarios can be traced to the tested intervention rather than to uncontrolled simulation changes. However, one fixed run only represents one possible realization of traffic behavior. Capturing variability in route choice and driving behavior therefore requires stochastic mechanisms and repeated simulations with different seeds [**SUMO_sumo**, **sumo_randomness**].

2.2.4 Alternatives to SUMO

Several other traffic simulation tools can serve as alternatives to SUMO, each with different features, advantages and limitations :

1. PTV-Vissim is a widely used microscopic traffic simulation software that also supports multi-modal transport systems. It offers a broad set of features comparable to those of SUMO. However, it is a proprietary tool that requires a commercial license, which can restrict its accessibility and limit the ability to implement custom modifications at the code level.

2. Aimsun is a commercial traffic simulation platform that supports microscopic, mesoscopic and macroscopic modeling. While it is used in various contexts, its commercial nature introduces licensing costs and reduces flexibility for developers who require deeper code-level access and customization.

3. CityFlow [**cityflow**] is an open-source microscopic simulator designed for efficient, large-scale urban traffic simulations, capable of handling thousands of intersections with high computational efficiency. Its architecture is highly optimized for speed and parallel execution, making it over twenty times faster than SUMO in large-scale benchmarks [**cityflow_rl**]. CityFlow is particularly popular in the field of artificial intelligence and reinforcement learning, serving as a benchmark for multi-agent adaptive traffic signal control and dynamic routing tasks. However, CityFlow remains limited compared to SUMO in certain areas: it offers fewer out-of-the-box features for conventional traffic simulation, has a less mature ecosystem and does not natively support the simulation of public transportation modes such as buses, trams, or trains, which restricts its suitability for comprehensive, multi-modal urban simulations.

2.2.5 Choice of SUMO for this thesis

SUMO was selected for this project based on the following criteria:

- **Open-source** : SUMO's open-source status allows users to freely modify, adapt and extend the software, making it suitable for research and academic use where flexibility and transparency are important. This degree of code-level access is not available in proprietary tools like PTV Vissim or Aimsun.
- **Community and documentation** : SUMO benefits from a large active community that contributes regularly to new features or bug fixes. In addition, it offers extensive documentation and a wide range of tutorials, which significantly lower the learning curve for new users.
- **Microscopic and multi-modal simulation** : SUMO provides detailed agent-based (microscopic) modeling of cars, public transport and pedestrians. While other simulators such as Vissim and Aimsun also support microscopic and multi-modal simulation, SUMO's implementation is open, which enables deeper customization and integration for research needs.
- **Integration and flexibility** :
  SUMO's design and command-line tools (like `netconvert` and `duarouter`) facilitates working on various data sources making it easy to connect with external components.

Commercial tools like PTV Vissim and Aimsun offer advanced graphical user interfaces and support, however, their proprietary nature limits customizability and long-term usability in research settings. CityFlow, while open-source and scalable, lacks SUMO's extensive documentation and wider range of traffic simulation features relevant for this project's objectives.

2.3 Stochastic simulation

In a traffic-management digital twin, uncertainty is part of the phenomenon being modeled rather than a secondary implementation detail. Traffic demand, vehicle insertion, lane changing, speed preferences and route choices all contain stochastic components and small changes in these components can propagate into visibly different congestion patterns [**digital_twin_transport**, **flow_twin**, **sumo_randomness**, **fhwa_microsim_2019**]. In the United States, the Federal Highway Administration (FHWA) therefore recommends that microsimulation alternatives be evaluated with multiple random seeds because stochastic model variation can affect the statistical validity of observed differences between alternatives [**fhwa_microsim_2019**].

Stochastic simulation treats selected inputs or model mechanisms as random variables and observes the distribution of outcomes over repeated replications [**law_kelton**, **number_of_runs_traffic**]. For this work, the decision output is therefore not a single deterministic delay value for each plan, but a set of paired delay, travel-time and impacted-user observations across simulation seeds. The evaluator uses these observations to estimate means, confidence intervals and ranking robustness.

2.3.1 Theoretical foundations of the Monte Carlo method

The Monte Carlo method estimates quantities of interest by repeated random sampling. If each simulation run is treated as one realization of an output metric, then repeated runs approximate the distribution of that metric under the model's stochastic mechanisms.

Law of Large Numbers and convergence. Let $X_1, \ldots, X_n$ be independent replications of an output metric X with finite mean μ. The sample mean $\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$ converges to μ as n increases. In the traffic-simulation setting, X_i is the metric produced by run i with a distinct random seed. This is why additional replications stabilize the estimated mean, provided the model configuration is otherwise held fixed [**law_kelton**, **number_of_runs_traffic**].

Confidence intervals. A sample mean alone does not communicate how much uncertainty remains after n runs. For a metric with sample standard deviation s, the usual small-sample interval for the mean is

$$\bar{X}_n \pm t_{0.975,n-1}\frac{s}{\sqrt{n}},$$

where $t_{0.975,n-1}$ is the Student-t critical value. This is the form used in the evaluator's 95% confidence intervals and is also the basis of the run-count formulas used in traffic microsimulation guidance [**law_kelton**, **txdot_number_runs**].

Paired design for plan comparison. A deviation plan and its baseline are compared under the same seed whenever possible. In simple terms, this means that the plan and the baseline are tested on the same simulated *traffic day*: the same random seed is used so that random effects such as vehicle insertion times or route-choice perturbations are aligned as much as possible. This matters because some seeds naturally create easier or harder traffic conditions. If a baseline run uses an easy seed and a deviation-plan run uses a difficult seed, the observed difference may come from the random draw rather than from the plan itself.

The paired design therefore compares each plan run only with the baseline run that uses the same seed. The estimator is applied to the run-by-run difference

$$\Delta^{(r)} = X^{(r)}_{\text{plan}} - X^{(r)}_{\text{baseline}}$$

rather than to two unrelated samples. Here, X can be a metric such as delay or mean travel time, and $\Delta^{(r)}$ asks a direct question for seed r: *how much did the plan change the metric relative to the comparable baseline?*. Preserving this run-by-run difference removes part of the noise caused by random simulation conditions and leaves a clearer estimate of the plan effect. The paired series is then summarized with the same confidence-interval machinery used for other stochastic simulation outputs.

2.3.2 Randomness in microscopic traffic simulation

SUMO documents stochasticity as a core mechanism for reproducing reality in a simulation scenario. The seed initializes the random-number generators and a fixed seed makes a run deterministic and reproducible. Changing the seed changes the random sequence used by stochastic model components.

Several SUMO mechanisms are directly relevant here. **Speed distributions** allow vehicles to draw heterogeneous speed preferences through the `speedFactor` parameter, SUMO states that this random value is selected when each vehicle is created and that a default speed distribution is active. **Car-following stochasticity** enters through model parameters such as `sigma`, which introduces random variation in driver speed behavior. **Departure-time variation** can be added with random offsets or randomized flows, which prevents unrealistically synchronized departures and better represents variable demand realization [**sumo_randomness**].

2.3.3 Determining the number of simulation runs

Choosing the number of simulation runs is difficult because the required number is not known in advance. It depends on the variability of the network, the metric studied, the desired precision and the performance gap between candidate plans. Stable networks or clearly separated alternatives may require only a few replications, whereas saturated networks or closely performing plans may require many more runs to obtain a reliable ranking.

The number of runs is therefore a methodological choice rather than a fixed property of traffic microsimulation. Existing guidance treats repeated random seeds as a way to capture stochastic variation and make uncertainty visible [**number_of_runs_traffic**, **fhwa_microsim_2019**, **txdot_number_runs**]. The goal is not to identify a universal run count, but to relate the run budget to the uncertainty remaining in the outputs.

In this thesis, the run count is treated as a precision-control choice rather than a universal constant. A small number of runs may be enough when demand is stable and candidate plans are clearly separated. More runs are needed when congestion is saturated or when the best plans have similar mean performance. The implementation therefore keeps all paired per-run observations, so that convergence plots and confidence intervals can be inspected after the simulations and the run budget can be extended when the recommendation is still unstable

2.4 Multiple-criteria decision analysis

Multiple-criteria decision analysis (MCDA) supports decisions where alternatives must be evaluated on several criteria that are not naturally reducible to one physical unit. In applied MCDA, the decision process is made explicit by defining criteria, assigning scores, choosing weights and showing how these elements produce a ranking [**mcda_manual**]. This reference is used here for that methodological structure, not as evidence about traffic behavior. The same structure fits deviation-plan evaluation: network delay, mean travel time, throughput feasibility and impacted-user burden are all relevant, but they do not express the same dimension of performance.

In this thesis, MCDA is not used to hide the trade-off inside one unexplained number. Instead, it is used to make the decision rule auditable. The evaluator first removes infeasible or dominated alternatives, then ranks the remaining plans with an additive score and finally tests whether the ranking is sensitive to the chosen weights.

2.4.1 Pareto dominance and non-dominated plan sets

Pareto dominance provides a weight-free screening rule. A feasible plan *a Pareto-dominates* a feasible plan b if a is no worse than b on every criterion and strictly better on at least one. Grierson gives the same formal condition for Pareto-optimal solutions in a multi-criteria minimization problem [**GRIERSON2008371**].

The non-dominated plans form the Pareto front. Restricting scoring to this set has a practical benefit: no choice of non-negative weights can justify selecting a plan that is uniformly worse than another feasible plan. This is particularly useful in deviation planning because a plan can look attractive on one metric while being worse on all others once paired simulation results are considered.

2.4.2 Additive weighted scoring

Additive weighted scoring is a simple and widely used aggregation method in multi-criteria decision analysis. Each alternative is evaluated on several criteria, whose values are first transformed onto a common scale. The overall score of alternative k is then computed as

$$S_k = \sum_q w_q z_{kq}, \tag{1}$$

where z_{kq} is the scaled performance of alternative k on criterion q and w_q is the weight assigned to that criterion. The weights are non-negative and usually sum to one.

This approach is valued for its transparency: the final ranking follows directly from the selected criteria and their weights [**mcda_manual**]. When all criteria are formulated as costs, lower scores indicate better alternatives. Its main limitation is that it allows compensation between criteria, meaning that poor performance on one criterion may be offset by good performance on another.

2.4.3 Weight sensitivity and robustness analysis

The main limitation of additive scoring is that the weights are not measured by the simulator. They express a value judgement about what the decision maker considers more important. For example, after all metrics have been normalized, assigning a larger weight to network delay than to impacted-user burden means that the evaluation is allowed to prefer a plan that saves more overall delay even if it affects more individual users. Another planner could reasonably make the opposite choice. The same simulation outputs can therefore lead to different rankings if the priorities are different. [**triantaphyllou_sensitivity_1997**].

In this work, weight robustness is estimated by drawing many admissible weight vectors from the simplex and recomputing the ranking. The sampling distribution is Dirichlet$(1, \ldots, 1)$, which is uniform over the weight simplex [**stan_dirichlet_simplex**]. The robustness score of a plan is the fraction of sampled weight vectors for which it ranks first. A high robustness score means that the recommendation does not depend strongly on the reference weight vector, whereas a low robustness score signals that the decision should be escalated to explicit stakeholder weighting.

3 Methodology

3.1 System overview and pipeline

The methodology follows the same order as the implemented workflow. The first step generates a synthetic, network-aware demand from the SUMO road network and the Traffic Analysis Zone (TAZ) layer described in Section 3.2.2. This demand is written as `trips.xml`: every trip has a departure time, an origin edge and a destination edge, while the full path is left open. The closure is then translated into candidate deviation plans. In the Monte Carlo stage, the same trip demand is routed repeatedly with `duarouter`, using randomized edge weights, before each deviation plan is applied and simulated in SUMO. The last step compares the plans using paired statistics and a multi-criteria decision rule. The functional organization of this workflow is summarized in Table 2, while Figure 3 shows the same pipeline as a data flow from inputs to final evaluation files.

3.1.1 Concepts and contribution boundary

The methods chapter combines standard transport concepts, external SUMO tools and code developed for this thesis. Table 1 defines the main terms before the individual stages are described.

Term	Meaning in this thesis
SUMO	*Simulation of Urban MObility*, the external microscopic traffic simulator used to execute the traffic runs. SUMO is used as a tool, not developed in this thesis.
`duarouter`[7]	SUMO's command-line router. It converts unrouted trips into explicit route files, this thesis uses its randomized edge-weight option to generate route-choice variation.
TAZ	*Traffic Analysis Zone*, a geographic zone used to aggregate origins and destinations before individual trips are instantiated on SUMO road edges.
OD matrix	*Origin-Destination* matrix. Entry (i, j) gives the number or share of trips from origin zone i to destination zone j.
Gravity model [**gravity_model**]	A standard demand-allocation model from transport planning. The thesis contribution is the synthetic implementation: zone-mass proxy, sampled TAZ-to-TAZ impedance and integer hourly OD matrices.
Deviation plan	A candidate detour path around the closed edge or edge sequence. The plan objects and route-rewriting policy are implemented in this thesis.
Baseline	The no-detour, no-closure condition for the same demand and Monte Carlo seed. Each candidate plan is compared with its paired baseline run.

Term	Meaning in this thesis
Monte Carlo run	One repeated simulation under a fixed trip set but a different routing seed and randomized edge-cost realization.
BFS	*Breadth-first search*, a standard graph search used here to find upstream candidate detour origins by hop depth.
MCDA	*Multi-criteria decision analysis*, the final ranking layer that combines time-loss delay, mean travel time and impacted-user delay after feasibility and Pareto screening.

Table 1: Key concepts used in the methodology and the boundary between reused methods/tools and thesis-specific implementation choices.

The core contribution is therefore not a new traffic simulator, a new gravity model or a new shortest-path algorithm. It is the integration of these standard components into a reproducible pipeline that generates synthetic demand, computes candidate detours, applies those detours to route files and evaluates them across paired stochastic SUMO simulations.

Table 2 identifies the four implemented modules, their role in the workflow and the artifacts they write for the following stage. The table should be read as the functional view of the system: it explains what each module is responsible for before the following subsections describe the internal methods in more detail.

Module	Main role	Main outputs
`scenario_generator`	Generates synthetic OD demand and unrouted trip records	`trips.xml`, OD matrices
`deviation_plan_maker`	Converts the network into a graph and calculates the deviation plan(s)	JSON of candidate deviation plans
`monte_carlo_simulation`	Re-routes trip demand and simulates baseline and detour conditions over multiple runs	`run_#` folders, `summary.json`
`evaluator`	Paired statistical analysis and multi-criteria ranking	`evaluation.json`, `evaluation.md`

Table 2: Functional organization of the pipeline.

Figure 3 complements the table with the execution order. The dashed box contains the three inputs shared by the workflow, the vertical arrows show the order in which modules are executed and the side boxes show the main files produced along the way.

[7] https://sumo.dlr.de/docs/CLI.html

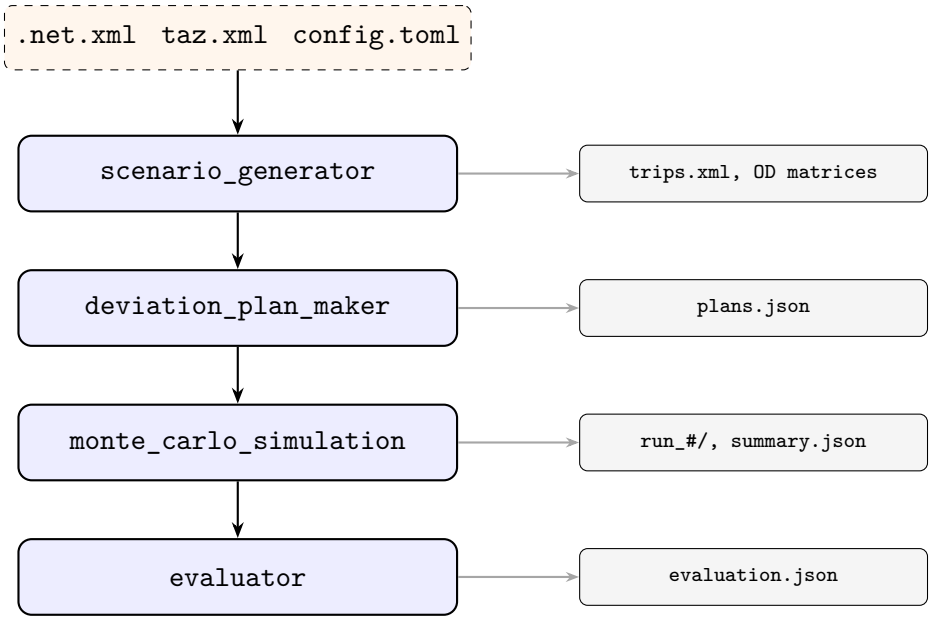


Figure 3: Four-module pipeline. Inputs (dashed box) flow top-to-bottom through the four modules. The scenario generator writes unrouted trips, explicit route files are produced later inside each Monte Carlo iteration by duarouter.

3.2 Scenario generation

The scenario generator creates the demand used by the rest of the pipeline. In this work, the demand must satisfy four constraints:

1. **Network-aware**: Origins and destinations are sampled from actual SUMO road edges.
2. **Zone-aware**: Demand is expressed at the TAZ level.
3. **Temporally structured**: The daily profile includes morning and evening peaks.
4. **Directional**: Peak periods can concentrate trips from residential zones toward employment zones and back.

This module is thesis code. It does not claim to estimate real Brussels demand from counts, surveys or mobile-phone traces. Instead, it constructs a controlled synthetic demand that is consistent with the available network files and is rich enough to test the detour-generation and evaluation pipeline. The standard modelling components used for this purpose are identified below, and the thesis-specific choices are the proxies, normalisations and sampling procedures used to turn them into reproducible SUMO inputs.

3.2.1 Module overview

The generation is split into three stages:

1. **Stage 1 - Demand generation**: construct 24 hourly Origin–Destination (OD) matrices using a gravity model [**gravity_model**] with an optional directional commute overlay.
2. **Stage 2 - Route library**: build a small set of candidate routes for each active OD pair using Yen's k-shortest paths algorithm [**yen_algorithm**].

3. **Stage 3 - Trip instantiation**: assign a departure time and valid endpoint edges to every trip. The full route is assigned later, inside the Monte Carlo stage.

3.2.2 Stage 1 : Demand generation

Stage 1 produces hourly OD matrices, not individual routes. The output of this stage is a set of zone-to-zone trip counts for each hour of the day. Individual departure times and SUMO edge endpoints are assigned only in Stage 3.

Traffic Analysis Zones

The spatial unit of the model is the *Traffic Analysis Zone* (TAZ), a standard construct in transport demand modelling that aggregates individual trip origins and destinations into discrete geographic areas. TAZs in SUMO are defined as polygons that can be associated with the road network. Each TAZ is defined by the set of SUMO road edges it contains.

A scalar *zone mass* m_i is assigned to each TAZ as the capacity of its drivable edge lengths :

$$m_i = \sum_{e \in \mathcal{E}_i} \ell(e) \cdot n(e) \tag{2}$$

where $\ell(e)$ is the length of edge e in meters and $n(e)$ is its lane count and $\mathcal{E}_i$ is the set of edges in TAZ i.

Equation (2) should therefore be interpreted as an infrastructure-based proxy for the relative importance of each TAZ, rather than as a calibrated measure of real travel demand. The formula only accounts for the total drivable road space inside a zone, through edge lengths and lane counts. The underlying assumption is that zones containing more and larger road infrastructure are likely to support more traffic activity and can therefore be assigned a larger synthetic production or attraction mass. This approximation is used because no calibrated socio-economic productions or attractions, such as population, employment, land use, or observed trip data are available for this synthetic setting. In a classical four-step transport model, these quantities would instead be estimated from socio-economic data [**modelling_transport**]. Here, lane kilometres[8] are used as a practical surrogate: they are directly available from the network and major road corridors generally carry more traffic than minor links. If observed productions and attractions become available, they should replace this proxy.

Temporal Demand Profile

The total daily demand N is distributed across the 24 hours of the simulation using a *two-Gaussian profile*:

$$p_h = \frac{a_{AM}\, g(h;\, \mu_{AM}, \sigma) + a_{PM}\, g(h;\, \mu_{PM}, \sigma)}{\displaystyle\sum_{h'=0}^{23} \Big[a_{AM}\, g(h';\, \mu_{AM}, \sigma) + a_{PM}\, g(h';\, \mu_{PM}, \sigma) \Big]}, \tag{3}$$

where

$$g(h;\, \mu, \sigma) = \exp\!\left(-\frac{(h-\mu)^2}{2\sigma^2} \right), \tag{4}$$

[8] A lane kilometre is one kilometre of road with one lane.

μ_{AM} and μ_{PM} are the morning and evening peak hours respectively and σ controls the temporal spread. The amplitudes a_{AM} and a_{PM} allow the two peaks to differ in magnitude. The default configuration uses $a_{AM} < a_{PM}$, representing a slightly larger afternoon peak, which is common in urban networks [**gaussian_book**].

The continuous profile p_h is converted into hourly trip counts n_h with the largest-remainder method. This keeps the total exactly equal to N while staying as close as possible to the continuous proportions.

Figure 4 illustrates the mathematical structure of the two-peak profile before integer rounding.

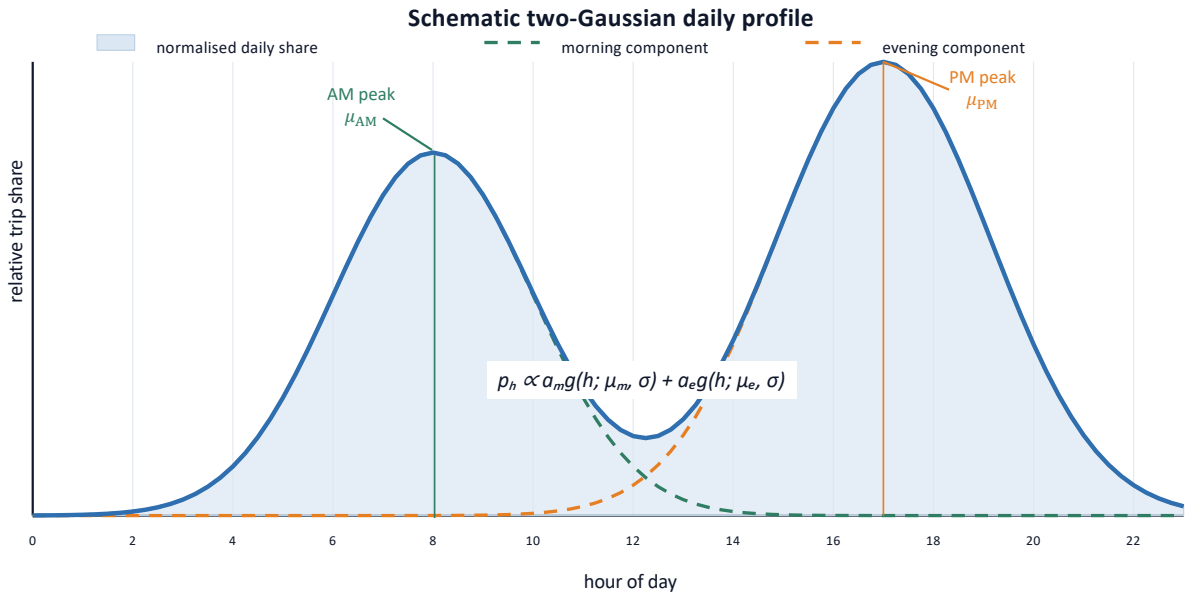


Figure 4: Illustrative two-Gaussian daily demand profile used to distribute the total daily demand across hours. The morning and evening components are centred on μ_{AM} and μ_{PM}, and the final hourly share is proportional to their weighted sum $a_{AM}g(h; \mu_{AM}, \sigma) + a_{PM}g(h; \mu_{PM}, \sigma)$.

Gravity OD model

The gravity model itself is not a new model introduced by this thesis. It is a standard transport-planning model used to distribute trips between origins and destinations as a function of origin mass, destination mass and travel impedance [**gravity_model**, **modelling_transport**]. What is specific to this work is its synthetic implementation: zone mass is approximated from lane kilometres, impedance is estimated from sampled shortest paths between TAZs and the resulting shares are converted into exact hourly integer OD matrices.

The spatial distribution of trips across TAZ pairs is governed by a *singly-constrained gravity model*. The **unnormalized** weight of the OD pair (i, j) is:

$$w_{ij} = m_i \cdot m_j \cdot \exp(-\beta\, c_{ij}), \quad i \neq j, \tag{5}$$

where c_{ij} is the travel-cost impedance between zones i and j (shortest path length in

meters) and $\beta \geq 0$ is the distance decay parameter[9].

Pairs with infinite or negative impedance receive zero weight. The normalized *OD share matrix* is then:

$$S_{ij} \;=\; \underbrace{\frac{m_i}{\sum_k m_k}}_{\text{origin share } O_i} \;\cdot\; \underbrace{\frac{m_j\,\exp(-\beta c_{ij})}{\sum_\ell m_\ell\,\exp(-\beta c_{i\ell})}}_{\text{conditional destination probability } p(j|i)} \quad , \tag{6}$$

so that, by construction, $\sum_j S_{ij} = m_i / \sum_k m_k$ and $\sum_{ij} S_{ij} = 1$.

For each hour h, the share matrix is scaled to the corresponding integer total n_h with the largest-remainder method, giving an exact integer OD matrix T^h with $\sum_{ij} T^h_{ij} = n_h$.

The model is *singly-constrained on the production side*. In trip-distribution terminology, productions are the trips generated by each origin zone, whereas attractions are the trips received by each destination zone. A production-side constraint therefore fixes the total fraction of trips that must leave each origin. In Equation (6), this production total is set by the origin mass share $m_i / \sum_k m_k$: after the destination probabilities for origin i are normalised, they must sum to one, so the whole row must sum to

$$\sum_j S_{ij} \;=\; \frac{m_i}{\sum_k m_k}$$

regardless of the destinations available from that origin. Equivalently, in an hour with n_h trips, origin i is assigned approximately $n_h m_i / \sum_k m_k$ outgoing trips before integer rounding.

The destination side is different. The term $m_j \exp(-\beta c_{ij})$ still affects how the trips from origin i are distributed across destinations: larger and closer destination zones receive higher conditional probability. However, no target total is imposed on the number of trips ending in destination j. The column sums $\sum_i S_{ij}$ are therefore left unconstrained and emerge from the aggregation of all origin rows. This is why the unnormalised weight $w_{ij} = m_i m_j \exp(-\beta c_{ij})$ can contain both origin and destination masses while the final model is still only constrained on the origin, or production, side.

In Wilson's balancing-factor formulation [**modelling_transport**], this corresponds to enforcing only the origin-side factor A_i and fixing the destination-side factor $B_j \equiv 1$. A doubly-constrained model would additionally enforce observed zonal attractions and would require both balancing factors to be recovered by iterative proportional fitting. Since zonal attractions are not observed in the present synthetic setting, enforcing them would add parameters that cannot be identified from the available data. For this reason, the singly-constrained formulation is used here: it relies only on the information present in the network (zone masses and impedances) and lets destination totals emerge from the destination masses and distance-decay term $\exp(-\beta c_{ij})$. Figure 5 gives a schematic view of the same directed OD relations in matrix form.

[9] A larger β means trips decay faster with distance/cost (people are more reluctant to travel far), a smaller β means the distribution is more spatially spread out. Wilson discusses this in the context of calibrating the model to observed data.[**gravity_model**]

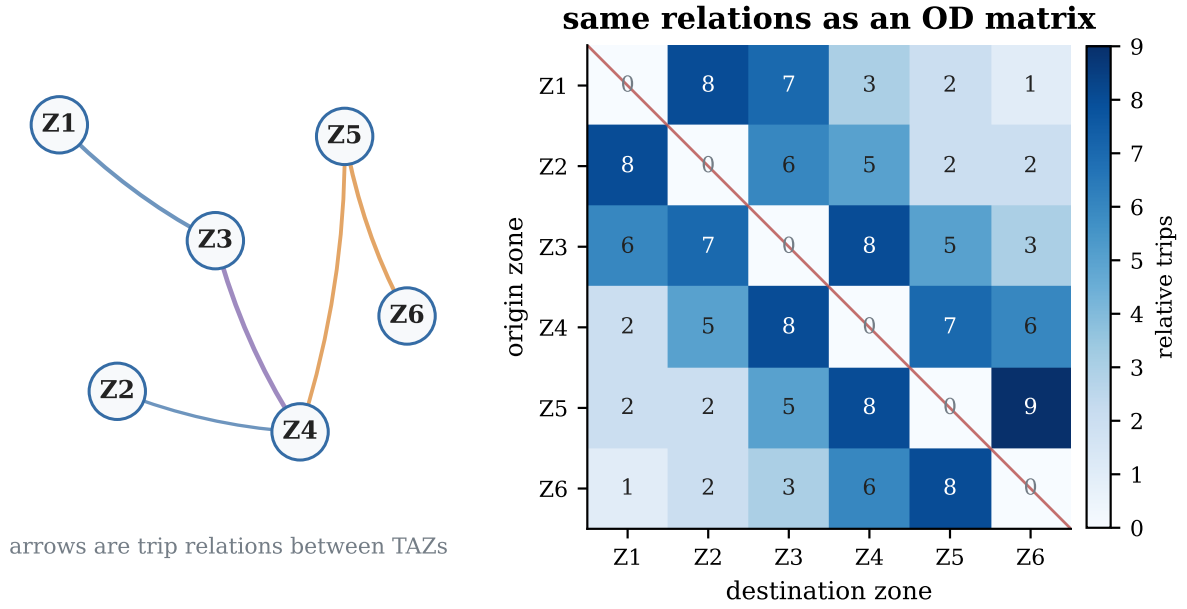


Figure 5: Schematic relationship between zone-to-zone trip relations and an OD matrix. Rows represent origin zones, columns represent destination zones, and each off-diagonal cell corresponds to the relative demand assigned to one directed OD pair. The diagonal is set to zero when intra-zonal trips are excluded.

Commute Overlay (Optional)

When residential weights r_i and employment weights e_i are available, the hourly OD shares can be tilted toward commuting patterns. Two additional gravity matrices are built: one for the morning peak (AM), from residential zones to employment zones and one for the evening peak (PM), in the reverse direction:

$$w_{ij}^{\mathrm{AM}} = m_i\, m_j\, f(c_{ij})\, r_i\, e_j, \tag{7}$$

$$w_{ij}^{\mathrm{PM}} = m_i\, m_j\, f(c_{ij})\, e_i\, r_j, \tag{8}$$

where $f(c_{ij}) = \exp(-\beta_c\, c_{ij})$ discounts longer or costlier trips. In the AM matrix, r_i and e_j increase flows leaving residential zones and arriving at employment zones. In the PM matrix, the same logic is reversed.

The two directional matrices are blended with the base OD share matrix through a time-varying mixture. For each hour h, the blended share is:

$$S_{ij}^{h} \;\propto\; S_{ij}^{\mathrm{base}} + \lambda\, g(h;\, \mu_m,\, \sigma)\, S_{ij}^{\mathrm{AM}} + \lambda\, g(h;\, \mu_e,\, \sigma)\, S_{ij}^{\mathrm{PM}}, \tag{9}$$

where $g(h;\, \mu,\, \sigma)$ is a Gaussian kernel centred on hour μ with spread σ. The parameter λ (`peak_weight`) controls how strongly the commute pattern modifies the background gravity model. Setting $\lambda = 0$ recovers the base model. After blending, the shares are normalised before being converted into integer trips. Figure 6 illustrates how the AM and PM terms tilt the base gravity share in opposite commute directions.

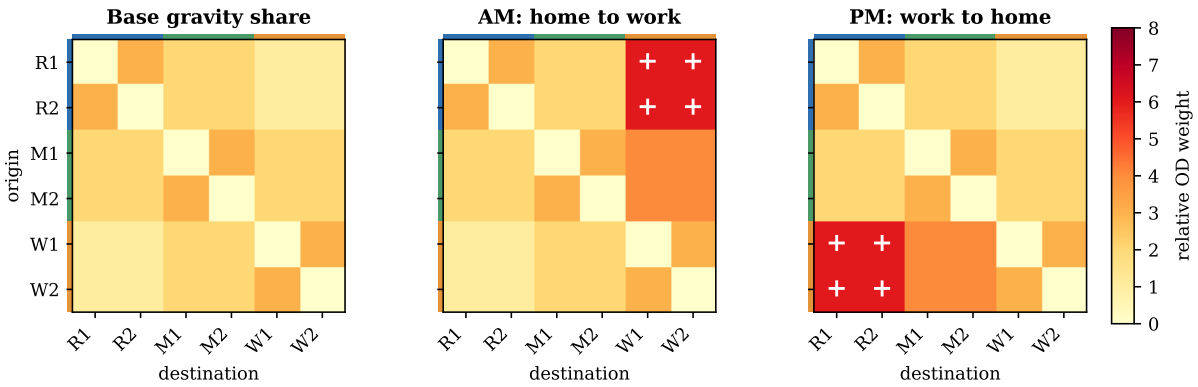


Figure 6: Illustrative commute overlay applied to the base gravity share. The AM term increases residential-to-workplace OD weights, while the PM term reverses the direction. In the implementation these directional matrices are blended with the base share through Equation 9 and then renormalised.

3.2.3 Stage 2 : Route library

Edge expanded network

Routing for the scenario generator uses an *edge-expanded* graph: SUMO road edges are the graph nodes and a directed arc connects edge u to edge v when SUMO allows the movement from u to v. The weight of this arc is the length of v, so a path cost corresponds to the sum of the edge lengths in the route.

This representation is convenient for two reasons. First, the generator has to sample SUMO edge endpoints and `duarouter` later expands those endpoints into complete routes. Staying at edge level keeps the generated demand close to SUMO's own trip format. Second, turning feasibility is already encoded by SUMO connections: if an arc exists, the turn is allowed. Internal junction edges[10] and edges that do not permit private-vehicle access are excluded when the graph is built.

Impedance Estimation

The gravity model needs a TAZ-to-TAZ impedance matrix. Computing exact all-pairs shortest paths would be expensive and would add precision that is not needed at this stage. Instead, for each directed zone pair (i, j), the generator samples a small number of origin-edge and destination-edge combinations from the two zones. The lowest observed shortest-path cost is used as the representative impedance. This is an approximation, but it gives the gravity model the spatial signal it needs: nearby and well-connected zones receive lower costs than distant or poorly connected ones.

k-Shortest Candidate Routes

For each active OD pair (i.e. $T_{ij}^h > 0$ for at least one hour h), a set of at most k candidate routes is computed as follows:

1. Sample s concrete (origin-edge, destination-edge) pairs, where edges are drawn propor-

[10] SUMO ids beginning with :

tionally to zone mass (Equation (2)).

2. For each pair, enumerate up to k shortest simple paths using Yen's algorithm [**yen_algorithm**]
3. Deduplicate across all samples by edge sequence, if the same sequence appears more than once, retain the lowest-cost instance.
4. Return at most k candidates sorted by cost.

3.2.4 Stage 3 : Trip Instantiation

Once the hourly OD matrices and the route library are available, every integer trip becomes a SUMO `trip` record. Two quantities are sampled: the departure time and a feasible candidate path. Only the first and last edges of that path are kept, because the final route is intentionally left to `duarouter` in the Monte Carlo stage.

Departure Time

Trips within each one-hour interval $[b, b+3600)$ are assumed to depart uniformly over that interval. For an OD cell containing n trips, departure times are drawn independently as:

$$t_1, \ldots, t_n \overset{\text{i.i.d.}}{\sim} \mathcal{U}(b,\ b+3600). \tag{10}$$

With only hourly totals available, this is the least committed assumption: there is no information that would justify placing trips at a more precise time within the hour.

Endpoint Selection

For an OD pair with candidate route set $\mathcal{R}_{ij}$, each trip first selects one candidate path uniformly at random:

$$P(r \mid \mathcal{R}) = \frac{1}{|\mathcal{R}|}, \quad r \in \mathcal{R}. \tag{11}$$

Only the first and last edges of the selected candidate are exported:

$$\texttt{from} = r_1, \qquad \texttt{to} = r_{|r|}. \tag{12}$$

This produces an unrouted `trips.xml` file. In this mode, the route library is used only to choose feasible endpoints, it is not a fixed route assignment. The full path choice is delegated to `duarouter` inside the Monte Carlo loop, where edge costs are randomized independently for each run.

The departure-time and endpoint draws use one seeded random-number generator. As a result, the generated `trips.xml` is reproducible. This matters because the Monte Carlo experiment is meant to vary routing and simulation seeds, not the underlying trip population.

3.3 Deviation plan computation

After demand generation, the next step is to define how the network can bypass the closure. The `deviation_plan_maker` takes the SUMO network and a user-selected closure

as input, then returns candidate detour paths around the disrupted segment. These plans are not evaluated at this stage, they are passed unchanged to the Monte Carlo simulation.

This module contains the main detour-construction contribution of the thesis: it converts a closure selected on the SUMO network into explicit detour-plan objects that can later be spliced into route files. The graph-search routines inside this module are standard algorithms. Dijkstra's algorithm is used for shortest paths, Yen's algorithm is used to enumerate loopless alternatives and breadth-first search (BFS) is used to discover upstream origins. The thesis contribution is how these algorithms are combined with closure-endpoint promotion, blocked-edge handling and depth-aware route rewriting.

Formally, let $G = (V, E, \ell)$ be a weighted directed graph in which V is the set of junctions, E the set of road edges and $\ell : E \to \mathbb{R}_{>0}$ an edge length function. Given a set of closed edges $E_c \subset E$ and a blocked set $B \supseteq E_c$, a *deviation plan* is defined as a tuple

$$\mathcal{D} \;=\; \Big(s,\;\; t,\;\; P,\;\; \ell(P),\;\; d\Big), \tag{13}$$

where $s \in V$ is the *origin* of this particular plan (the junction at which the detour begins) and $t \in V$ is the detour target, while $P = (e_1, \ldots, e_m)$ is a loopless path in $G' = (V,\, E \setminus B)$ from s to t and $\ell(P) = \sum_{i=1}^{m} \ell(e_i)$ is its total geometric length in meters. The integer $d \in \mathbb{N}_0$ is the *breadth-first search (BFS) depth* of s relative to the *effective detour source* returned by the endpoint computation of Section 3.3.2. In the common case this effective source coincides with the upstream junction of the first closed edge $s_0 = \mathit{from}(e_1^c)$. If that junction has no usable alternative exit, the endpoint computation moves the source further upstream and adds the traversed prefix edges to the blocked set B. In the upstream method, the origin s can differ from one plan to another: the effective source, t and B are shared, but individual plans may start further upstream. When $d = 0$, the plan starts at the effective source and acts as the shallowest fallback. Plans with $d > 0$ start further upstream and are tried first during multi-plan route rewriting.

3.3.1 Junction-level graph representation

The `deviation_plan_maker` operates on a *junction-level* directed graph, in contrast to the edge-expanded representation used by the scenario generator in Section 3.2.3. Here nodes correspond to SUMO junctions and arcs correspond to road edges: an arc (u, v) exists for every edge whose `from` attribute is junction u and whose `to` attribute is junction v. Arc weights default to the geometric length of the edge in meters. An alternative weight can be loaded from a SUMO `edgedata.xml` file, in which case values are averaged across all measurement intervals and the resulting mean is stored as the weight, if the field encodes speed rather than travel time, the reciprocal is stored instead so that shorter weighted distances correspond to faster edges.

The graph is built directly from the `.net.xml` file. The implementation keeps three dictionaries (`edges`, `outgoing` and `incoming`) that provide $O(1)$ lookup of any edge record and $O(\deg(v))$ enumeration of a node's neighbours[11]. These structures are sufficient for the routing operations used in this module.

[11] Here, $\deg(v)$ denotes the number of outgoing or incoming edges adjacent to node v, depending on the direction of the enumeration. Thus, neighbour enumeration is linear in the local branching factor of the queried junction, not in the total number of edges in the network.

3.3.2 Detour endpoint computation

Given a sequence of closed edges $E_c = (e_1, \ldots, e_n)$, the natural candidate detour source is the upstream junction of e_1 and the candidate target is the downstream junction of e_n. However, if the source junction has only one outgoing edge (the first closed edge itself), no detour can diverge from it, the same problem arises symmetrically at the target if it has only one incoming edge.

To handle this, Algorithm 1 walks upstream from the source and downstream from the target until it reaches junctions with at least two outgoing (resp. incoming) edges. Every intermediate edge encountered during these walks is added to the blocked set, so the router cannot reuse it as a shortcut.

Algorithm 1 FindDetourEndpoints

Require: closed edge sequence E_c, graph G
Ensure: source s, target t, extended blocked set B

start with s and t at the upstream and downstream ends of the closure
initialise B with the closed edges
while s has only one way out (the closure itself) **do**
    step one junction further upstream and add the traversed edge to B
end while
while t has only one way in **do**
    step one junction further downstream and add the traversed edge to B
end while
return s, t, B

Figure 7 illustrates the procedure on a small graph in which the natural endpoints s_0 and t_0 are both dead ends and the walks promote s and t as the actual detour endpoints.

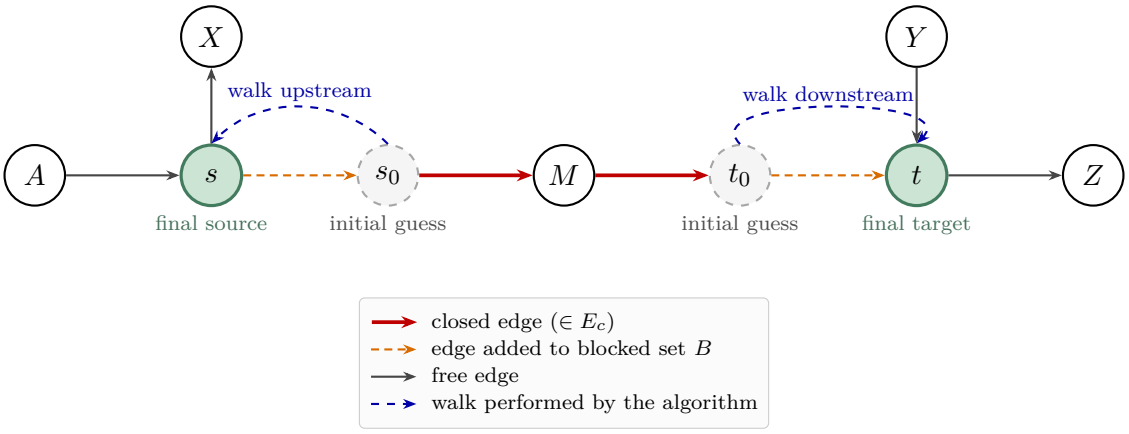


Figure 7: Detour endpoint computation on an illustrative graph. The closure $E_c = (s_0 \to M,\ M \to t_0)$ is shown in red. Because s_0 has only one outgoing edge (the closure itself) and t_0 has only one incoming edge, neither can serve as a detour endpoint. The algorithm walks one junction upstream from s_0 to s (which has the additional outgoing edge $s \to X$) and one junction downstream from t_0 to t (which has the additional incoming edge $Y \to t$). The two traversed edges (orange, dashed) are appended to the blocked set B so they cannot be reused by the router as shortcuts around the closure.

3.3.3 Candidate generation

Two candidate-generation methods are implemented. Both start from the same endpoint computation, but they use it differently. Yen's method keeps one detour source s and enumerates the k best loopless paths from s to t, all of its plans have depth $d = 0$. The upstream method starts from the same effective source, then searches backwards up to $d_{\max}$ hops. Each upstream node becomes a possible plan origin and the shortest path from that origin to t is retained if it exists. Yen's method is the default in the end-to-end pipeline. The upstream method is used when the experiment explicitly needs detours that begin before the closure point.

Yen's k-shortest loopless paths.

With source s, target t and blocked set B fixed, the first method calls Yen's k-shortest loopless paths algorithm [**yen_algorithm**] to enumerate up to k alternative routes from s to t that avoid every edge in B.

As described in Algorithm 2, Yen's algorithm extends Dijkstra's shortest-path search by iteratively generating *spur paths*. Let P_1 be the shortest path returned by Dijkstra. At each subsequent iteration i, the algorithm considers every prefix of the previously accepted path P_{i-1} as a potential root segment. For each root, it temporarily blocks the outgoing arc that was used by any already-accepted path sharing that root, then runs Dijkstra from the spur node (the last node of the root) to t. The resulting spur path is concatenated with the root to form a complete candidate. All candidates are stored in a min-heap keyed by total cost and the minimum is promoted to P_i. A global signature set prevents the same edge sequence from appearing twice. Heap entries carry a monotonically increasing tie-breaking counter, so that two paths with identical cost are ordered by insertion time rather than by a potentially fragile lexicographic comparison of node lists.

Algorithm 2 High-level pseudocode for YenKShortestPaths, which enumerates up to k loopless detour paths while avoiding blocked edges.

Require: graph G, source s, target t, integer $k \geq 1$, blocked set B
Ensure: list $\mathcal{P}$ of up to k diverse shortest paths from s to t avoiding B

$P_1 \leftarrow$ shortest path from s to t avoiding B
$\mathcal{P} \leftarrow [P_1]$
for $i \leftarrow 2$ **to** k **do**
    candidates $\leftarrow \varnothing$
    for each prefix of the previously accepted path P_{i-1} **do**
        temporarily block the next arc taken by any already-accepted path that shares this prefix
        find the shortest path from the prefix's endpoint to t
        store *prefix + new path* as a candidate
    end for
    pick the cheapest unseen candidate as P_i and append it to $\mathcal{P}$
    stop if no candidate remains
end for
return $\mathcal{P}$

Upstream search.

The upstream method is based on the fact that vehicles approaching a closure may arrive from several directions, not from one junction only. A reverse BFS from s records every reachable upstream node u and its depth $d(u)$ up to $d_{\max}$. The source s itself is kept at depth 0, so the method still contains a fallback. For each discovered origin, Dijkstra is run from that origin to t while avoiding the blocked set B.

The returned plans are sorted by depth descending and then by total length. This ordering is used later by the route rewriter: it tries the most upstream applicable plan first, then falls back to shallower ones. Algorithm 3 summarizes this reverse search, path computation and sorting step.

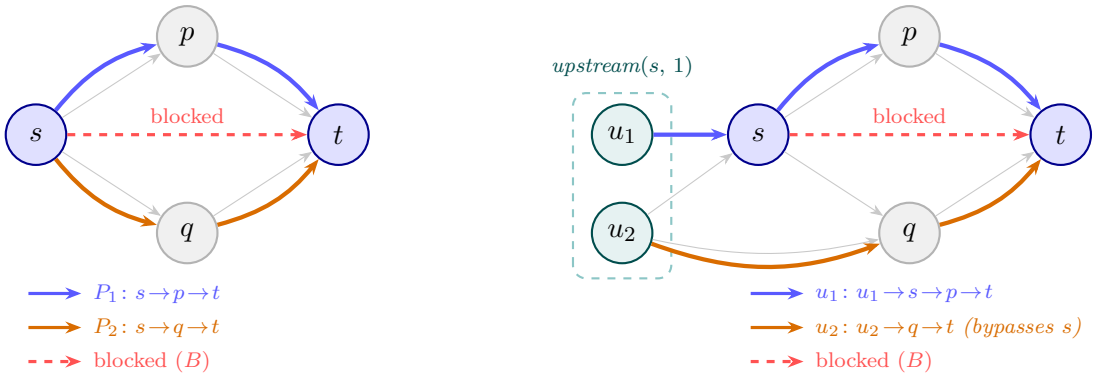
Algorithm 3 High-level pseudocode for UpstreamDeviationPlans, which searches upstream origins and sorts feasible detours by depth and length.

Require: graph G, endpoints s, t, blocked set B, max depth $d_{\max}$
Ensure: deviation plans sorted by depth (desc), then length (asc)

reverse-BFS from s up to $d_{\max}$ hops, tagging each reachable node u with its depth $d(u)$
$\mathcal{P} \leftarrow [\,]$
for each upstream origin u found (including s at depth 0) **do**
    compute the shortest path π from u to t avoiding B
    if π exists and is not a duplicate **then**
        save $\mathcal{D}(\pi,\, d(u))$ as a candidate plan in $\mathcal{P}$
    end if
end for
sort $\mathcal{P}$: deeper origins first, shorter paths first within each depth
return $\mathcal{P}$

where $\mathcal{D}(\pi, d)$ denotes the deviation plan wrapping path π with BFS depth d.

Figure 8 illustrates the two methods on the same toy network: nodes s and t are the detour endpoints, p and q are intermediate junctions and the direct edge $s \to t$ belongs to the blocked set B.



(a) Yen's k-shortest paths ($k = 2$)

(b) Upstream multi-origin ($d = 1$)

Figure 8: Comparison of the two candidate generation methods on a toy network. In both cases the edge $s \to t$ is blocked (red dashed). *(a)* Yen's algorithm produces $k = 2$ loopless paths, both originating from s: P_1 takes the upper corridor through p and P_2 the lower corridor through q. *(b)* The upstream BFS at depth $d = 1$ identifies two origins $\{u_1, u_2\}$ (teal). u_1 routes through s and then takes the upper corridor, while u_2 holds a direct connection to q, so its optimal path bypasses s entirely, a candidate that Yen's method, being anchored at s, would never generate.

Each accepted path is wrapped in a `DeviationPlan` object. The object records the plan identifier, its rank within the k-shortest list, the ordered sequences of junction and edge identifiers and the total geometric length in meters (computed by summing the `length_m` attribute of every edge on the path).

No post-filtering is applied at this point. Plans are not removed because of a maximum detour-length ratio and near-duplicates are not removed by edge-set overlap. This keeps the planning stage simple and avoids discarding a route before it has been tested in simulation. Two routes can share most of their edges and still behave differently if they diverge near a bottleneck or pass through a different sequence of signalised intersections. The Monte Carlo evaluation is used later to separate useful candidates from poor ones.

3.3.4 Route rewriting

Detours are applied by rewriting route files before SUMO starts, rather than by using SUMO's dynamic *rerouters* [**sumo_rerouter**]. This keeps the experiment static and inspectable: `duarouter` already provides route-choice variation through randomized edge weights and the final route file can be checked before the simulation.

After each Monte Carlo call to `duarouter`, the rewriter creates one `routes_{plan_id}.xml` file per candidate plan. Only vehicles that traverse a closed edge are modified. For each affected vehicle, candidate plans are tried from deepest to shallowest and the first plan whose origin appears upstream of the closure in that vehicle's route is selected (Algorithm 4). Yen plans have only the evaluated depth-0 candidate, upstream plans also carry the depth-0 fallback.

The selected plan $\mathcal{D}^*$ is inserted by replacing the route segment from $\mathcal{D}^*.s$ to the last closed edge with the detour edge sequence $\mathcal{D}^*.P$. This is a splice in the vehicle's edge list.

Algorithm 4 High-level pseudocode for DEPTHAWAREREWRITE, which selects the deepest applicable detour and splices it into an affected route.

Require: vehicle route R, candidate plans $\mathcal{D}$ sorted by depth (desc), closed edge set B_c
Ensure: rewritten route R'

1: **if** R does not traverse any edge in B_c **then**
2: **return** R *// route unaffected by the closure*
3: **end if**
4: pick the deepest plan $\mathcal{D}^*$ whose origin s lies on R upstream of the closure *// shallower plans provide deterministic fallback*
5: splice $\mathcal{D}^*.P$ into R, replacing the segment that runs from the edge departing $\mathcal{D}^*.s$ up to the last closed edge
6: **return** the spliced route

The baseline route file is left unchanged, each candidate is evaluated through its own rewritten copy.

Figure 9 illustrates this route-list splice for one affected vehicle.

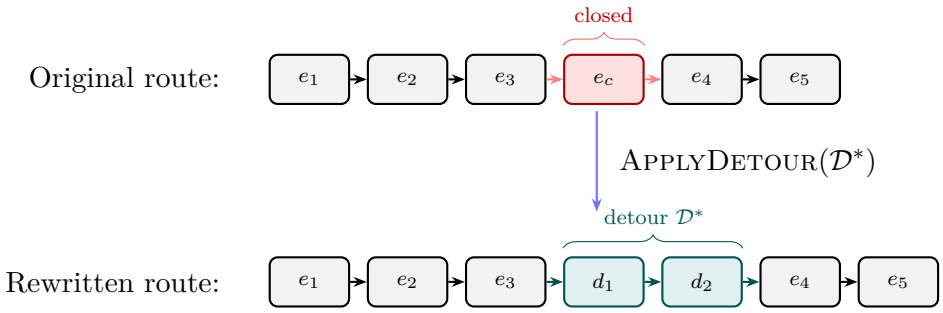


Figure 9: Route rewriting for one affected vehicle. The closed edge e_c (red) is replaced by the selected detour sequence (d_1, d_2) (teal), while the unaffected prefix and suffix are preserved.

3.4 Monte Carlo Simulation

The Monte Carlo stage turns the fixed demand and the candidate detours into repeated SUMO observations. In this stage, the demand stays fixed while the route assignment changes. The scenario generator gives the simulation an unrouted `trips.xml` file and each Monte Carlo iteration routes that same trip set again with `duarouter`, a different seed and randomized edge weights.

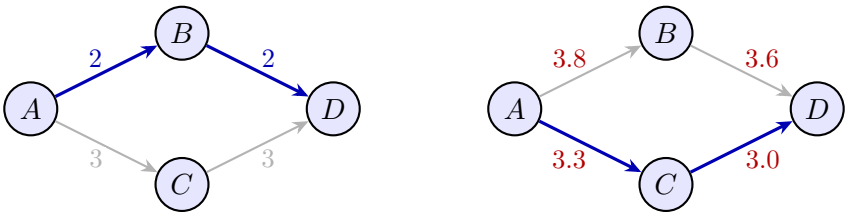
Here, a Monte Carlo run means one repetition of the same experiment under a different routing random seed. The baseline in each run is the route realization without applying any detour plan or closure treatment. Candidate plans are then applied to a copy of that same baseline route file. This paired design means that a plan is compared against the exact route-choice realization from which it was produced, rather than against an average from other seeds.

3.4.1 Experimental design

The experiment uses a **paired stochastic-routing design**. The trip population is fixed, while the route realization changes from run to run. Let $\mathcal{T}$ be the `trips.xml` file produced by the scenario generator. For Monte Carlo iteration r, the simulation first computes the baseline route file

$$\mathcal{R}_0^{(r)} = \textsc{DuaRouter}\big(\mathcal{T},\, G,\, r,\, \alpha\big), \tag{14}$$

where G is the SUMO network and α is the upper bound passed to `--weights.random-factor`. With this option, `duarouter` perturbs edge costs by drawing multipliers in $[1, \alpha]$. This does not change the trip origin or destination, it changes only how attractive each road segment appears during route computation. In human terms, this represents a driver who still wants to reach the same destination but may slightly adapt the chosen path after observing or anticipating traffic on particular streets. Different iterations can therefore produce different paths for the same trip endpoints. Figure 10 illustrates this mechanism on a small two-route graph.



No perturbation ($\times 1$) *With perturbation* ($\alpha = 2$)

Figure 10: Effect of `-weights.random-factor` on route choice. Both graphs share the same network and origin–destination pair $A \to D$. **Left**: nominal edge costs, the router selects $A \to B \to D$ (cost 4) over $A \to C \to D$ (cost 6). **Right**: each cost is multiplied by a random factor drawn from $[1, \alpha]$, the top path becomes costlier (7.4) than the bottom path (6.3), so the router now chooses $A \to C \to D$. Across Monte Carlo iterations, different draws of these multipliers yield different route realizations for the same trip set.

For every candidate deviation plan $\mathcal{D}_k$, the deterministic rewriter is then applied to that same baseline route file. The baseline and all plan conditions are paired within the same run: they share the same trip set, the same route realization before applying the closure treatment and the same SUMO seed.

Meanwhile all available metrics are collected and saved for later evaluation. For each iteration r, the raw time-loss difference

$$\delta_k^{(r)} \;=\; D_k^{(r)} \;-\; D_0^{(r)} \tag{15}$$

measures the effect of plan $\mathcal{D}_k$ relative to the no-closure baseline under the same route-choice and SUMO random seed. The estimator

$$\bar{\delta}_k = \frac{1}{R} \sum_{r=1}^{R} \delta_k^{(r)} \tag{16}$$

compares plans after much of the run-level noise shared by the baseline and plan conditions has been removed.

3.4.2 Per-run routing and detour application

There is no plan-specific route pre-computation outside the Monte Carlo loop. The only fixed demand input is `trips.xml`. In each iteration, `duarouter` writes `run_#/routes.xml`. This is the baseline route realization for that seed. The route rewriter then creates the `routes_{plan_id}.xml` files in the same run directory.

After each plan file is written, the code checks that the closed edge no longer appears in any rewritten route. It also compares the baseline and plan files vehicle by vehicle to count how many routes changed. These statistics are stored per run, since the affected vehicles can change when `duarouter` produces a different baseline route set.

3.4.3 Iteration structure

Each Monte Carlo iteration consists of routing, deterministic route rewriting, SUMO execution and metric extraction. Algorithm 5 states the procedure formally.

Algorithm 5 Pseudocode for one MonteCarloIteration, including routing, route rewriting, SUMO execution and metric extraction.

Require: network file G, trip file $\mathcal{T}$, closed edge e_c, candidate plans $\{\mathcal{D}_1, \ldots, \mathcal{D}_K\}$, edge-endpoint map $\mathcal{E}$, run index r, simulation window $[b, e]$, random factor α

Ensure: per-condition metric dictionary for this iteration

$\mathcal{R}_0^{(r)} \leftarrow \textsc{DuaRouter}(\mathcal{T}, G, r, \alpha)$ ▷ baseline route realization

$M_0 \leftarrow \textsc{RunSUMO}(G, \mathcal{R}_0^{(r)}, r, b, e)$

results[baseline] $\leftarrow M_0$

for $k \leftarrow 1$ **to** K **do**

  $\mathcal{R}_k^{(r)} \leftarrow \textsc{ApplyDetourToRoutes}(\mathcal{R}_0^{(r)}, \mathcal{D}_k, e_c, \mathcal{E})$

  validate that $\mathcal{R}_k^{(r)}$ contains no closed edge e_c

  $s_k^{(r)} \leftarrow \textsc{CountRerouted}(\mathcal{R}_0^{(r)}, \mathcal{R}_k^{(r)})$

  $M_k \leftarrow \textsc{RunSUMO}(G, \mathcal{R}_k^{(r)}, r, b, e)$

  results[$\mathcal{D}_k$] $\leftarrow M_k \cup s_k^{(r)}$

end for

return results

DuaRouter invokes `duarouter` with the fixed trip file, `-weights.random-factor` and the run seed. RunSUMO writes a minimal SUMO configuration file that specifies the network, the route file, the simulation window $[b, e]$ and the same seed r, invokes `sumo` as a subprocess and parses the resulting `tripinfo.xml` output into the metrics of Section 3.4.4. Each condition runs in its own subdirectory (`run_r/baseline/`, `run_r/plan_k/`), keeping outputs isolated.

Figure 11 summarizes the same iteration as a data flow from fixed inputs to per-run baseline and plan outputs.

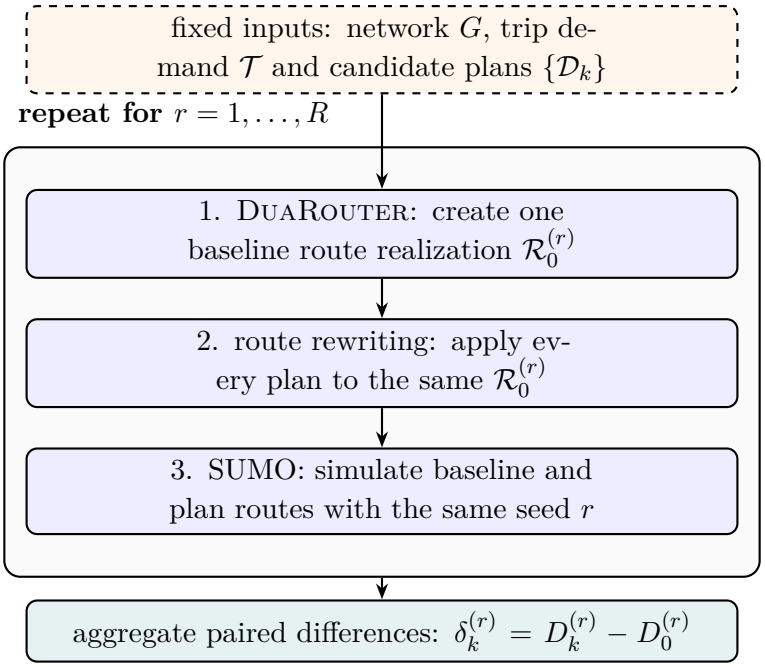


Figure 11: Monte Carlo protocol. The demand and candidate plans are fixed, each seed produces one baseline route realization, applies all detours to that realization and simulates paired baseline-plan conditions.

3.4.4 Performance metrics

Each `tripinfo.xml` file is parsed into a metric vector $M_c^{(r)}$ for condition c and run r. SUMO is configured with `tripinfo-output.write-unfinished`, so completed trips can be separated from unfinished or vaporized records. Aggregate performance is computed only on completed trips. The unfinished count is kept as a diagnostic.

Time-loss delay is the primary network-efficiency metric, but it is not the only assessment criterion. The Monte Carlo stage stores time-loss delay, mean travel time, completed trips, unfinished trips and the number of vehicles whose route changed after applying a plan. The evaluator then derives additional impacted-user metrics from the route and `tripinfo.xml` files.

- **Total time-loss delay** D (vehicle-hours): the sum of SUMO `timeLoss` over completed trips, converted from seconds to hours,

$$D = \frac{1}{3600} \sum_{v \in \mathcal{V}_{\text{done}}} \lambda_v, \tag{17}$$

  where λ_v is the vehicle's SUMO `timeLoss`. This is the excess travel time relative to SUMO's ideal traversal of the driven route, so it captures both stopped waiting and moving slowdowns.

- **Mean travel time** $\bar{\tau}$ (minutes): the arithmetic mean of completed trip durations,

$$\bar{\tau} = \frac{1}{60\,|\mathcal{V}_{\text{done}}|} \sum_{v \in \mathcal{V}_{\text{done}}} \tau_v, \tag{18}$$

  where τ_v is the door-to-door duration of trip v in seconds.

- **Completed and unfinished trips**: the number of vehicles that arrived within the simulation window and the number of records that did not. A high unfinished count flags a simulation window, demand, or topology problem that should be inspected before interpreting a plan.
- **Rerouted vehicles**: for each plan and run, the number and percentage of vehicles whose route sequence differs from the run's baseline route file. This value is computed after `duarouter`, so it can vary across Monte Carlo iterations.

For plan comparison, the evaluator computes paired differences such as $\delta_k^{(r)}$ in Equation (15) run by run. These raw paired observations are preserved in the per-run `results.json` files and are not replaced by only top-level averages.

3.4.5 Convergence and number of runs

The number of runs is fixed by the user. If more precision is needed, the same output directory can be extended using a resume mechanism, which continues from the next missing run identifier.

3.5 Evaluation and decision support

The last step is the evaluation of the simulated plans. At this point no new traffic is generated and no additional SUMO simulations are run. The task is to take the repeated outputs from the Monte Carlo experiment and compare the candidate detours in a way that is fair, reproducible and useful for the final discussion. The key point is to keep the paired structure of the experiment: for each Monte Carlo seed, the evaluator has one baseline result and one result for each candidate deviation plan. Each plan is compared against the baseline from the same run, not against an unrelated average.

The assessment is deliberately multi-metric. Total time-loss delay is treated as the primary network-efficiency metric, but it is not the only criterion used to judge a plan. Mean travel time, completed-trip feasibility and impacted-user delay are all kept in the evaluation, and the final ranking is produced by a multi-criteria decision analysis (MCDA) layer rather than by a single time-loss number.

In practice, the evaluation is organised around three questions:

- How much each plan changes the network-level performance compared with the baseline.
- Whether the plan remains feasible, in particular whether it avoids reducing the number of completed trips.
- Among the feasible plans, which alternatives still look reasonable after Pareto screening and a sensitivity check on the decision weights.

3.5.1 Choice of evaluator metrics

The evaluator does not try to prove that one detour is universally best. It separates the comparison into quantities that answer different planning questions:

- **Total time-loss delay** measures the network-level congestion burden created or

removed by a plan. It is the primary efficiency metric because it captures both queues and slow movement relative to the vehicle's ideal route traversal.

- **Mean travel time** measures the average trip burden for completed vehicles. It complements time-loss delay because a plan can reduce congestion on the chosen route while still making paths longer, or conversely add local congestion while shortening many trips.
- **Completed and unfinished trips** are used as a feasibility diagnostic. A plan must not appear good merely because more vehicles fail to finish within the simulation window and are therefore absent from completed-trip averages.
- **Fraction of impacted users delayed** focuses on the vehicles whose route actually changed. Network averages can hide a plan that performs well overall but imposes most of the cost on the directly affected users.

These metrics are also practical: they are available for every Monte Carlo run from SUMO's `tripinfo.xml` output and from the route comparison step, so the same definitions can be applied consistently to every candidate plan. The statistical tests reported by the evaluator are diagnostic checks on the paired effects, they are not the ranking rule. The final decision layer uses effect sizes, feasibility, Pareto dominance and weight sensitivity.

3.5.2 Comparable run set

The first operation is to identify the runs that can actually be compared. The evaluator scans the Monte Carlo output directory for `run_#/results.json` files and keeps a run only when it contains the baseline and all candidate plans listed in `summary.json`. Runs with missing plan outputs are left out of the paired analysis, but they are still recorded in the metadata so that the exclusion is visible.

This step avoids comparing one plan over more replications than another. After the filtering, every paired statistic is computed on the same run identifiers for every plan. If no complete run is available, the evaluator stops instead of producing a ranking based on incomplete evidence.

3.5.3 Paired network-level statistics

For every plan $\mathcal{D}_k$ and every retained Monte Carlo run r, the basic quantity is the difference between the plan and the baseline from the same run. For the primary network-level metric, this gives

$$\Delta D_k^{(r)} = D_k^{(r)} - D_0^{(r)}, \tag{19}$$

where $D_0^{(r)}$ is the baseline time-loss delay from run r and $D_k^{(r)}$ is the corresponding value after applying plan $\mathcal{D}_k$. For mean travel time, the same construction is used:

$$\Delta \bar{\tau}_k^{(r)} = \bar{\tau}_k^{(r)} - \bar{\tau}_0^{(r)}. \tag{20}$$

The sign convention is kept simple: a positive paired difference means that the plan worsened the metric in that run and a negative value means that it improved it. The full

vector of paired differences is kept, not only the average, because the consistency of an effect matters as much as its mean.

For a generic paired-difference vector $x_1, \ldots, x_n$, the evaluator reports

$$\bar{x}, \quad s, \quad \min(x), \quad \max(x), \quad \tilde{x}, \tag{21}$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and $\tilde{x}$ the median. The mean estimates the expected effect of the plan over the seed distribution, the standard deviation describes run-to-run variability and the median is kept as a robust centre when a few runs are extreme.

The confidence interval answers a different question: how precisely the Monte Carlo experiment has estimated the mean paired effect. Because the true standard deviation of the paired effects is unknown and must be estimated from the n replications, the normal critical value is replaced by the Student-t critical value. The Student-t distribution has heavier tails than the normal distribution, so it gives wider intervals when only a small number of runs is available. The evaluator uses the standard 95% level, corresponding to a two-sided 5% error rate, as a conventional compromise between intervals that are too narrow to be reliable and intervals that are too wide to be useful. The interval is computed as

$$\bar{x} \pm t_{0.975,\, n-1} \frac{s}{\sqrt{n}}, \tag{22}$$

when at least two runs are available. In repeated experiments satisfying the usual independence assumptions, this procedure would contain the true mean effect in about 95% of such intervals, it does not mean that there is a 95% probability that this particular realized interval contains the truth.

Two diagnostic hypothesis tests are reported for each paired metric. Both ask whether the observed paired effects are compatible with a zero-effect baseline. The paired one-sample t-test checks the null hypothesis

$$H_0 : \mathbb{E}[\Delta M_k] = 0, \tag{23}$$

where ΔM_k is the paired effect of plan k on the metric being tested. It is most appropriate when the paired differences are approximately normally distributed. A Wilcoxon signed-rank test is also reported as a non-parametric companion. It ranks the absolute non-zero paired differences, attaches the original signs and checks whether positive and negative ranks are balanced around zero. This makes the check less dependent on the normality assumption, which is useful for traffic simulation outputs with small samples or occasional extreme runs.

3.5.4 Feasibility gate

Before plans are ranked, they pass through a completed-trip feasibility check. A candidate is considered feasible only if it completes at least as many trips as the baseline in every retained run:

$$C_k^{(r)} \geq C_0^{(r)} \qquad \forall r \in \{1, \ldots, n\}, \tag{24}$$

where $C_k^{(r)}$ is the number of completed trips under plan $\mathcal{D}_k$. The reason for this rule is straightforward: a detour should not look better simply because more vehicles failed to

arrive before the end of the simulation window. Plans that fail the gate are still shown in the report, but they are not allowed into the Pareto screening or weighted score.

3.5.5 Impacted-vehicle metrics

Network-level metrics are necessary, but they can hide what happens to the drivers who are directly affected by the closure. For this reason, the evaluation also considers an impacted vehicle set. When route files are available, a vehicle is counted as impacted for a plan if its edge sequence differs between the baseline route file and the plan-specific route file:

$$\mathcal{I}_k = \{\, v : R_{k,v} \neq R_{0,v} \,\}. \tag{25}$$

If route-difference information is not available, the evaluator can fall back to the vehicles whose baseline route crosses the closed edge. This is less precise, but it still keeps the analysis focused on users directly connected to the disruption.

For each retained run, the impacted-user analysis compares only vehicles that are present and completed in both the baseline and plan `tripinfo.xml` files. It reports the total additional travel time for impacted users,

$$\Delta T_{k,\mathcal{I}}^{(r)} = \sum_{v \in \mathcal{I}_k} \left(\tau_{k,v}^{(r)} - \tau_{0,v}^{(r)} \right), \tag{26}$$

the mean effective speed over their trajectories and the fraction of impacted users who are delayed:

$$\phi_k^{(r)} = \frac{|\{\, v \in \mathcal{I}_k : \tau_{k,v}^{(r)} > \tau_{0,v}^{(r)} \,\}|}{|\mathcal{I}_k|}. \tag{27}$$

The mean of $\phi_k^{(r)}$ across runs is used as one of the three decision criteria. Lower values are preferred, because they mean that a smaller share of directly affected users experiences a travel-time increase.

3.5.6 Multi-criteria decision analysis

The final ranking is built in three steps, following the general MCDA idea of making criteria and weights explicit rather than hiding them in an implicit single score [**mcda_manual**]. First, infeasible plans are removed by the completed-trip gate of Section 3.5.4. Second, the remaining plans are screened for Pareto dominance using three cost criteria:

$$\left(\overline{\Delta D_k}, \overline{\Delta \bar{\tau}_k}, \overline{\phi_k} \right). \tag{28}$$

A feasible plan a Pareto-dominates a feasible plan b if it is no worse on all three criteria and strictly better on at least one. Dominated plans are removed before the weights are applied. This prevents the final ranking from selecting a plan that is worse than another available plan on all measured dimensions.

Third, the plans left on the Pareto front are ranked with a normalized additive score. For each criterion q, values are min-max normalized over the Pareto front:

$$z_{kq} = \frac{x_{kq} - \min_j x_{jq}}{\max_j x_{jq} - \min_j x_{jq}}, \tag{29}$$

with $z_{kq} = 0$ when all surviving plans have the same value on criterion q. The composite score is then

$$S_k = \sum_q w_q z_{kq}, \tag{30}$$

where all criteria are costs and lower scores are better. The reference configuration gives equal weight to network time-loss delay, mean travel time and the fraction of impacted users delayed. This should be read as a neutral starting point, not as a claim that the three dimensions are inherently equally important.

Because the weights contain judgement, the evaluator also checks how sensitive the ranking is to them. It samples weight vectors uniformly from the three-criterion simplex using a Dirichlet$(1, 1, 1)$ distribution. Here a weight vector is a triple (w_1, w_2, w_3) with non-negative entries that sum to one. For each sampled weight vector, the evaluator recomputes the ranking. The report then shows how often each Pareto-efficient plan ranks first, its mean rank, score quantiles and pairwise win probabilities. A plan that wins under the reference weights and also remains strong under many sampled weights is more convincing than a plan that only wins under a narrow weighting assumption.

4 Results

This chapter gives a visual and quantitative readout of the complete pipeline. The first part uses the *city_tiny* test scenario, a toy network generated for this thesis rather than an extract of a real city. It serves as a controlled test bed: the objective is to show that the implemented workflow produces coherent demand, route alternatives, Monte Carlo outputs and a reproducible plan ranking. The final part repeats the same protocol on a small road-only SUMO network around Flagey in Brussels. That second experiment is still synthetic in its demand calibration, but it checks that the pipeline also works on a real neighbourhood topology with irregular streets and OpenStreetMap-derived junction geometry. Figure 12 gives the neutral network view used as the starting point for the first experiment.

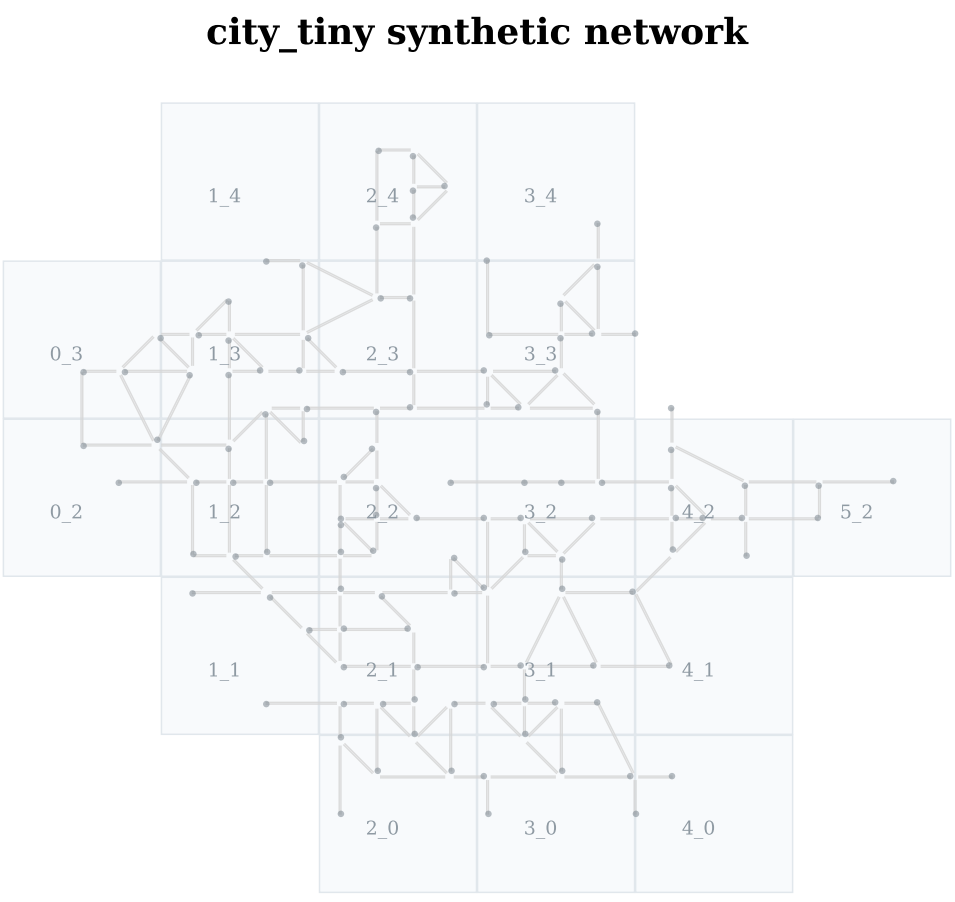


Figure 12: Synthetic `city_tiny` toy network created for this thesis and used as a controlled test case for the draft experiment. This small network is useful for testing the full pipeline before moving to real street topologies. TAZ boundaries are shown in light grey and the road graph is drawn without closures or detours to give a neutral view of the study area.

4.1 Experimental setup

The scenario generator produced 15,200 vehicle definitions from 24 hourly OD matrices. The closure is directed edge `172`, which connects origin junction `2` to destination junction `171`. The two manually defined candidate deviation plans also run from junction `2` to junction `171`, but they use different replacement corridors. The manually defined plans were evaluated over 30 Monte Carlo runs, while the Yen and upstream batches were evaluated over 10 runs in this draft experiment. The same evaluator is applied to all plan families, but the number of replications differs at this draft stage. Figure 13 illustrates the overall pipeline and Table 3 summarises the main results for the manually-defined plans.

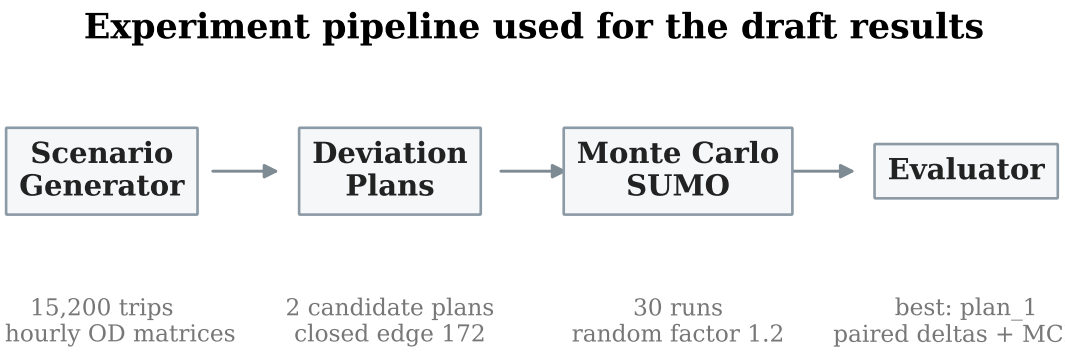


Figure 13: Concrete result-producing pipeline for the draft experiment. The same generated demand is reused across all Monte Carlo runs, variation comes from stochastic route assignment before each plan-specific simulation.

Plan	Delay	Δ delay	Δ travel	Rerouted
Baseline	63.12	–	–	–
`plan_1`	69.47	6.36	0.090	1,166.7
`plan_2`	71.49	8.37	0.151	1,166.7

Table 3: Main Monte Carlo aggregates over 30 runs. Delay and delta delay are in vehicle-hours, delta travel is in minutes per completed vehicle. Deltas are plan minus baseline, where the baseline is the unclosed-network reference.

For reference, the deterministic candidate-generation stage is inexpensive on this toy network. A simple wall-clock benchmark on the development machine, averaged over 200 repetitions, loaded the 364-edge SUMO graph in 13.8 ms, generated the 8 Yen candidates in 5.3 ms after graph loading, and generated the 17 upstream candidates in 2.0 ms after graph loading. These timings exclude the SUMO Monte Carlo simulations, which dominate the runtime and scale with the number of replications and candidate plans. The evaluator post-processing for the 30-run, two-plan manual result set took about 23 s because it reparses the stored XML outputs.

Step	Output	Mean wall-clock time
Load SUMO network graph	364 directed edges	13.8 ms
Yen generation, $k = 8$	8 candidate plans	5.3 ms
Upstream generation, depth 4	17 candidate plans	2.0 ms
Evaluator post-processing	30 runs, 2 plans	≈ 23 s

Table 4: Indicative computation times for the draft experiment. Candidate generation times exclude SUMO simulation and are measured after the input data were already present on disk.

4.2 Manually-defined plans

The test network contains 364 directed road edges, 364 lanes, 120 junctions and 20 Traffic Analysis Zones (TAZs), for a total of 30.82 lane-kilometres. Figure 14 shows the spatial scale of the experiment. The closed edge sits in the central part of the network and both candidate plans route traffic around this local disruption.

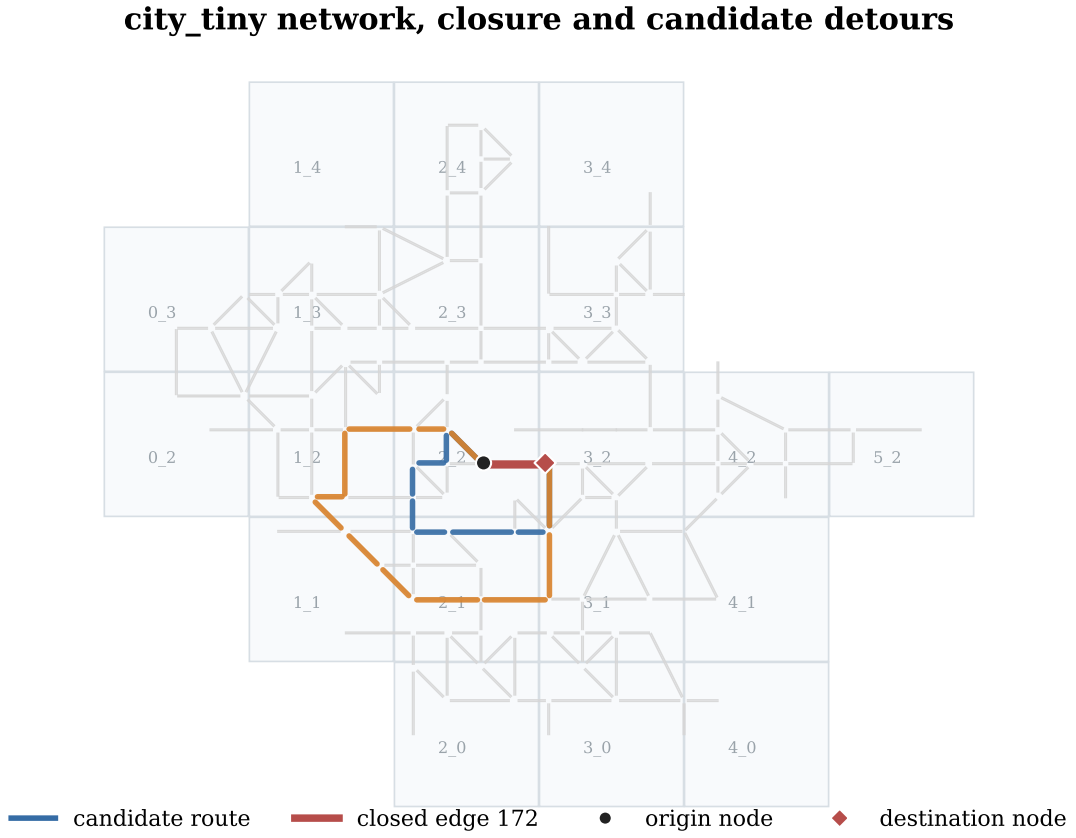


Figure 14: Synthetic network, TAZ grid, closed edge and the two candidate detours. The grey lines are the full road network, red marks the closed edge, blue and orange mark `plan_1` and `plan_2` and the common detour endpoints are origin junction `2` and destination junction `171`.

Plan	Origin	Destination	Edges	Length (km)	Interpretation
`plan_1`	`2`	`171`	9	0.635	Short manually defined corridor close to the closure.
`plan_2`	`2`	`171`	12	1.160	Wider manually defined corridor that sends traffic farther around the closure.

Table 5: Explicit origin and destination of the two manually defined candidate plans. Both replace closed edge `172`, which itself runs from junction `2` to junction `171`.

The generated demand has the intended two-peak structure as illustrated in Figure 15. The largest hourly volume occurs at 17:00, with 1,328 departures, which is consistent with the configured evening-dominant profile. The daily OD matrix is also spatially concentrated rather than uniform: zone `2_1` is the largest origin zone (1,553 trips) and the largest destination zone (1,611 trips) in this synthetic configuration, as suggested by Figure 16.

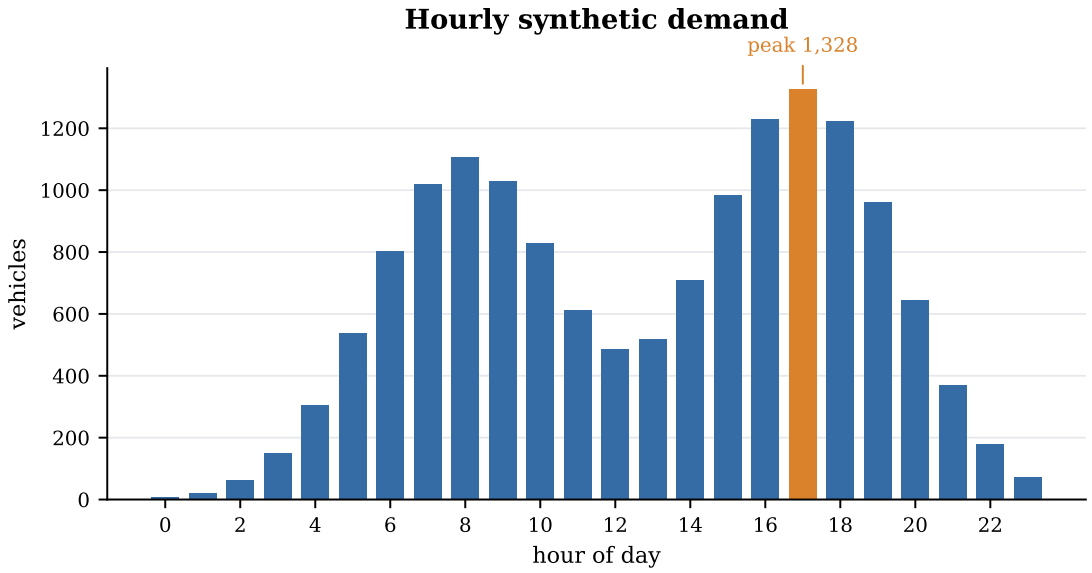


Figure 15: Hourly trip demand generated for the experiment. The evening peak dominates the synthetic day, while off-peak periods remain lightly loaded.

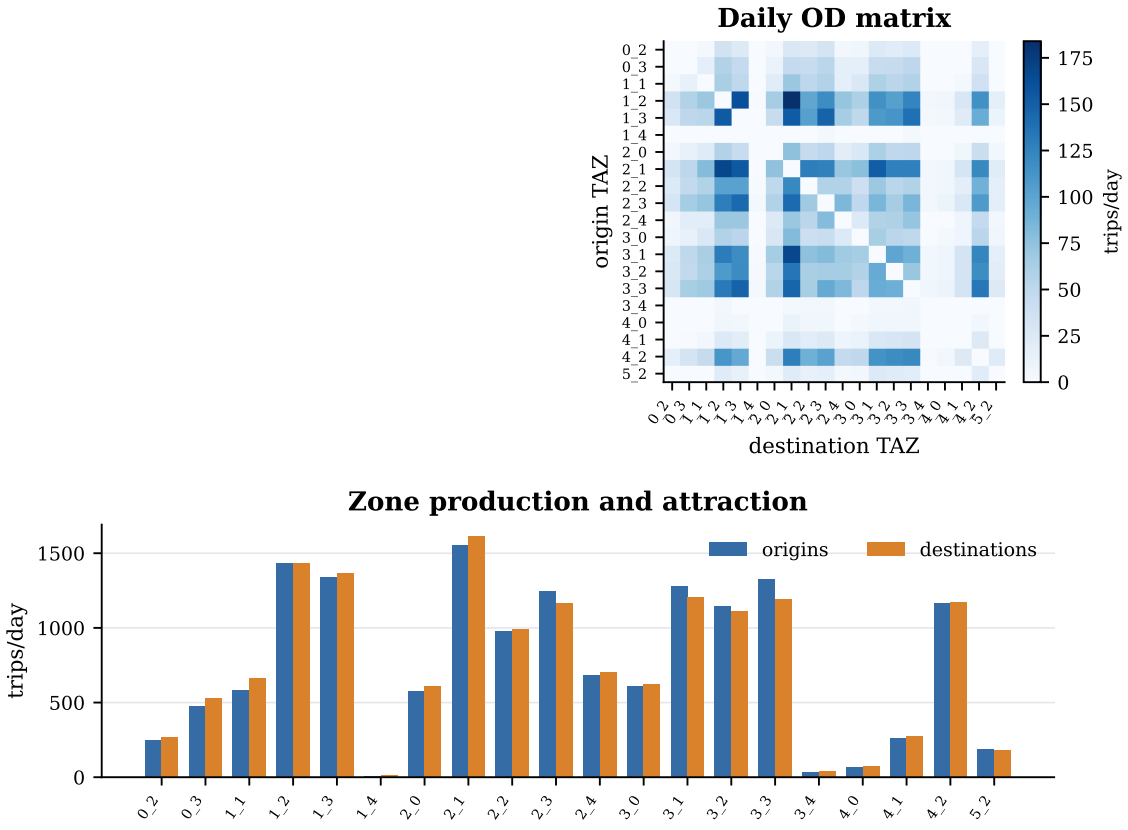


Figure 16: Daily OD structure and zone-level activity. The heatmap gives the full daily OD matrix, while the lower panel compares each zone's generated origins and destinations.

The two tested alternatives differ mainly in path length and spatial spread. `plan_1` uses 9 detour edges, covers 0.635 km and remains closer to the closed link. `plan_2` uses 12 detour edges, covers 1.160 km and makes a wider diversion. In the simulated demand, both alternatives reroute the same average number of vehicles, about 1,167 vehicles per run, corresponding to 7.67% of the daily demand. This makes the comparison easy to read: the difference between plans is driven by the replacement path, not by a different affected-vehicle count.

4.2.1 Evaluator verdict

The evaluator assessed both plans on three criteria aggregated over the 30 Monte Carlo runs: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip (in minutes) and the flow-weighted congestion proxy ϕ_{net}. Both plans are feasible, meaning neither reduces the number of completed trips below the baseline.

`plan_1` Pareto-dominates `plan_2` on every criterion: its mean incremental delay is 6.36 vh against 8.37 vh for `plan_2`, the per-vehicle travel-time penalty is +0.090 min against +0.151 min and the congestion proxy ϕ_{net} is 0.392 against 0.403 as illustrated in Table 6. Because `plan_2` is Pareto-dominated, no weighting of the three criteria can make it the preferred choice. The robustness test confirms this: drawing $N = 20{,}000$ weight vectors uniformly from the weight simplex (Dirichlet$(1, 1, 1)$ sampling), `plan_1` ranks first in every single draw (100% frequency). The evaluator therefore recommends `plan_1` as the

best manually-defined deviation plan for this closure. Figure 17 shows a visual summary of the decision.

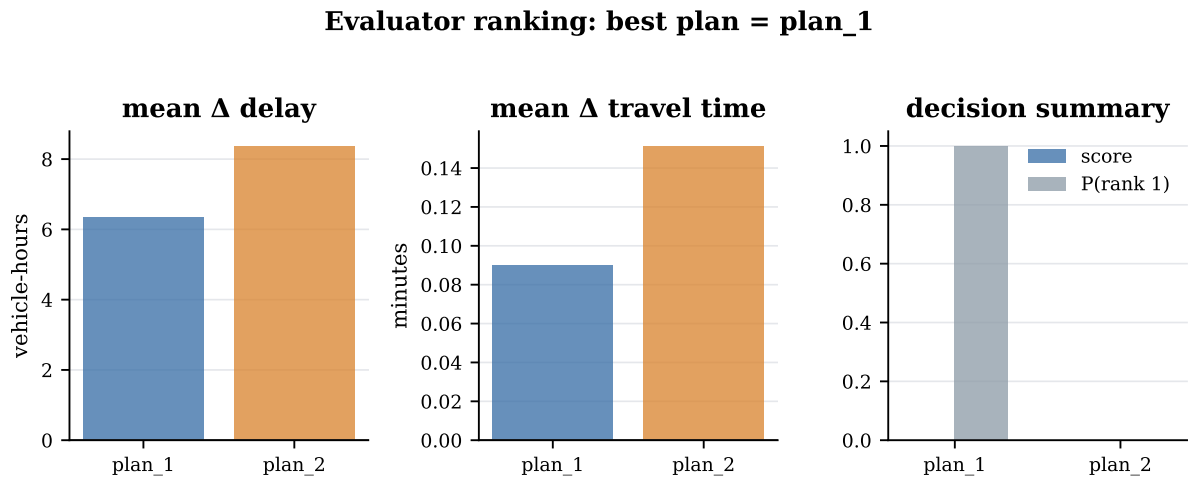


Figure 17: Evaluator decision summary for the two manually-defined candidate plans. `plan_1` Pareto-dominates `plan_2` on all three criteria and is ranked first across all sampled weight vectors. Both alternatives replace closed edge `172` from origin junction `2` to destination junction `171`.

4.3 Yen-based plans

While the two candidate plans above were defined manually, the pipeline also offers an automated route-generation mode based on Yen's k-shortest loopless paths algorithm. Applied to edge `172`, the algorithm first resolves the effective detour endpoints. Starting from the nominal source junction of the closed edge, the graph walks upstream until it finds node `2`, the first junction with more than one outgoing edge. It similarly resolves the downstream target to node `171`. The closed edge and any approach edges encountered during this endpoint walk are added to the blocked set so they cannot be used as shortcuts.

Yen's algorithm then enumerates up to k loopless paths from node `2` to node `171` with the blocked set applied. The 8 paths share the same detour origin at node `2` and differ only in the sequence of edges used to reach node `171`. They are therefore all depth-0 plans. The two shortest paths (`yen_01` and `yen_02`) each use 8 edges and cover 0.559 km, they are equal-length but traverse distinct edge sequences. Two alternatives use 10 edges and cover 0.627 km, while the remaining four use 9 edges and cover 0.635 km. The longest Yen paths are therefore about 14% longer than the two shortest ones, but still shorter than the wider manual `plan_2`. This is the first place where the automated approach adds value: starting from the same closure endpoints as the expert-defined plans, it exposes two shorter options and several nearby corridor variants. Figure 18 shows the eight alternatives separately in a two-column by four-row grid. Showing one route per panel makes the small branch differences between equal-length alternatives clearer than a single overlay.

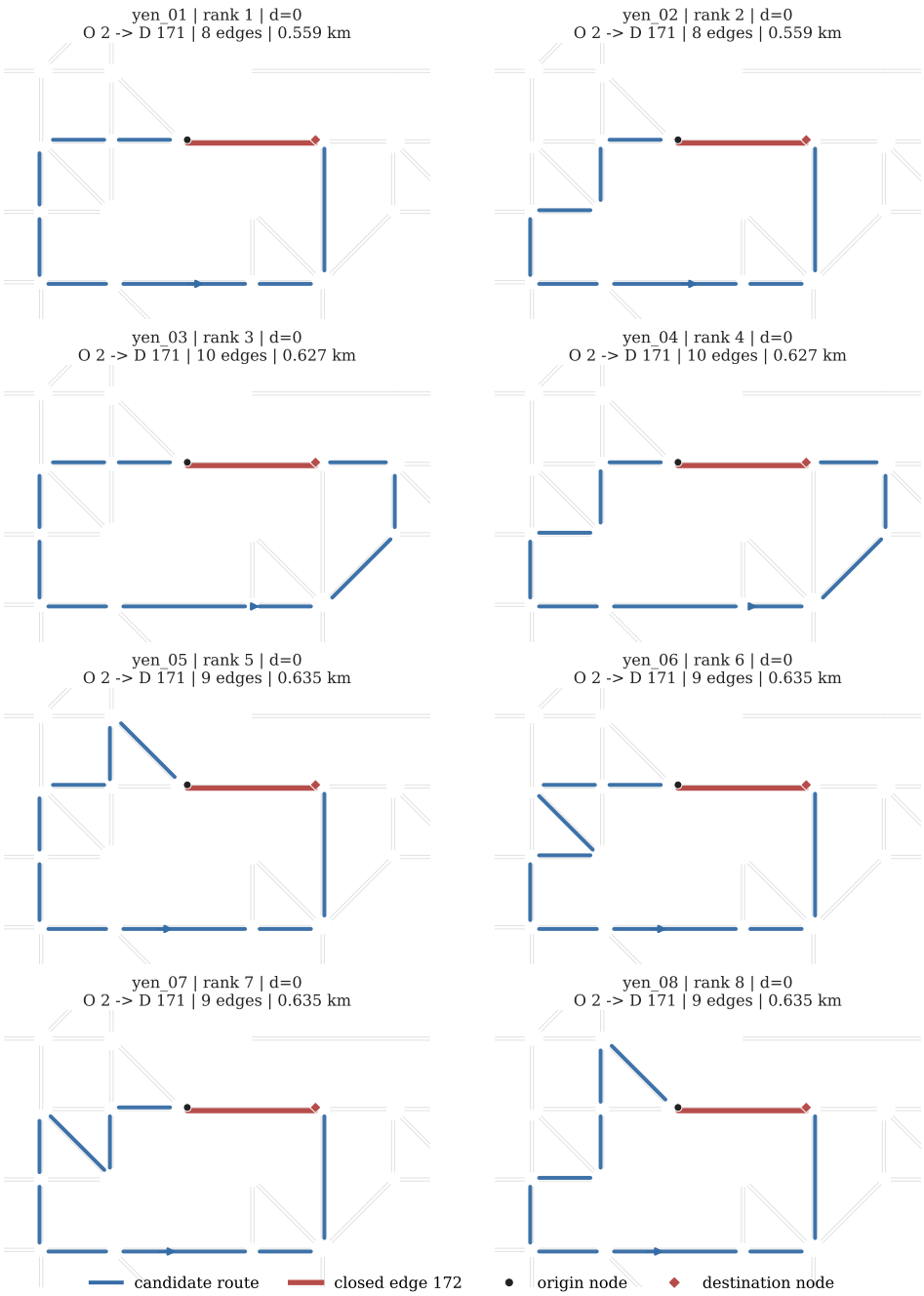


Figure 18: The eight candidate deviation plans produced by Yen's k-shortest paths algorithm for the closure of edge `172`. Each panel isolates one route: blue indicates the candidate route, red indicates the closed edge and the panel title gives the route's origin and destination nodes. All Yen alternatives use origin junction `2` and destination junction `171`.

The Yen output provides a cost-ordered menu of alternatives all anchored at the immediate source junction of the closure. The two equal-length top paths offer planners a choice of parallel corridors at minimal added distance, while the longer paths trade extra kilometres for greater spatial distribution of rerouted demand.

4.3.1 Evaluator verdict

The evaluator assessed the 8 Yen-generated plans on the same three criteria as before: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip and the flow-weighted congestion proxy ϕ_{net}. All Yen plans are feasible, meaning none reduces the number of completed trips below the baseline.

The Pareto screening leaves only two efficient alternatives: `yen_01` and `yen_06`. `yen_01` gives the lowest delay and travel-time penalty, with a mean incremental delay of 4.35 vh and a mean added travel time of +0.070 min per completed trip. However, `yen_06` has the lowest congestion proxy, with $\phi_{\text{net}} = 0.378$ against 0.380 for `yen_01`. This means that `yen_01` is not a strict Pareto winner: it is better on the two time-based criteria, while `yen_06` is slightly better on the congestion criterion.

The remaining six plans are Pareto-dominated. In particular, `yen_01` dominates `yen_02`, `yen_03`, `yen_04`, `yen_05`, `yen_07` and `yen_08`, so these alternatives are removed before weighted scoring. With equal reference weights, the additive score ranks `yen_01` first. The robustness test confirms this preference but not unanimously: drawing $N = 20{,}000$ weight vectors uniformly from the weight simplex (Dirichlet$(1, 1, 1)$ sampling), `yen_01` ranks first in 74.95% of draws, while `yen_06` ranks first in 25.05% of draws. The evaluator therefore recommends `yen_01` as the best Yen-generated deviation plan for this closure, while identifying `yen_06` as a relevant alternative if the congestion proxy is given very high priority.

4.4 Upstream-based plans

The upstream method extends the depth-0 Yen logic by performing a reverse breadth-first search (BFS) from the closure's source node and running one Dijkstra query to the target node per discovered upstream origin. Each resulting plan is tagged with the BFS depth of its origin, indicating how many junctions upstream of the closure the rerouting begins. A depth-d plan therefore intercepts traffic before it has reached the approach segment, which is relevant when queues propagate backwards under congested conditions.

Applied to edge `172` with a maximum BFS depth of 4, the reverse search discovered 17 reachable origin nodes across depths 0 through 4, each yielding a unique Dijkstra path to destination junction `171`, for a total of **17 upstream deviation plans**. The depth distribution is: 1 plan at depth 0 (`upstream_17`, the universal fallback at junction `2`), 2 plans at depth 1 (junctions `1` and `12`), 4 plans at depth 2 (junctions `6`, `8`, `22`, `28`), 4 plans at depth 3 (junctions `15`, `157`, `159`, `165`) and 6 plans at depth 4 (junctions `34`, `42`, `178`, `190`, `197`, `201`).

Detour lengths span from 0.351 km (`upstream_01`, depth 4, 4 edges, junction `42`) to 0.794 km (`upstream_06`, depth 4, 11 edges, junction `201`). Figure 19 then shows the same upstream candidates as isolated alternatives, following the presentation used for the Yen candidates.

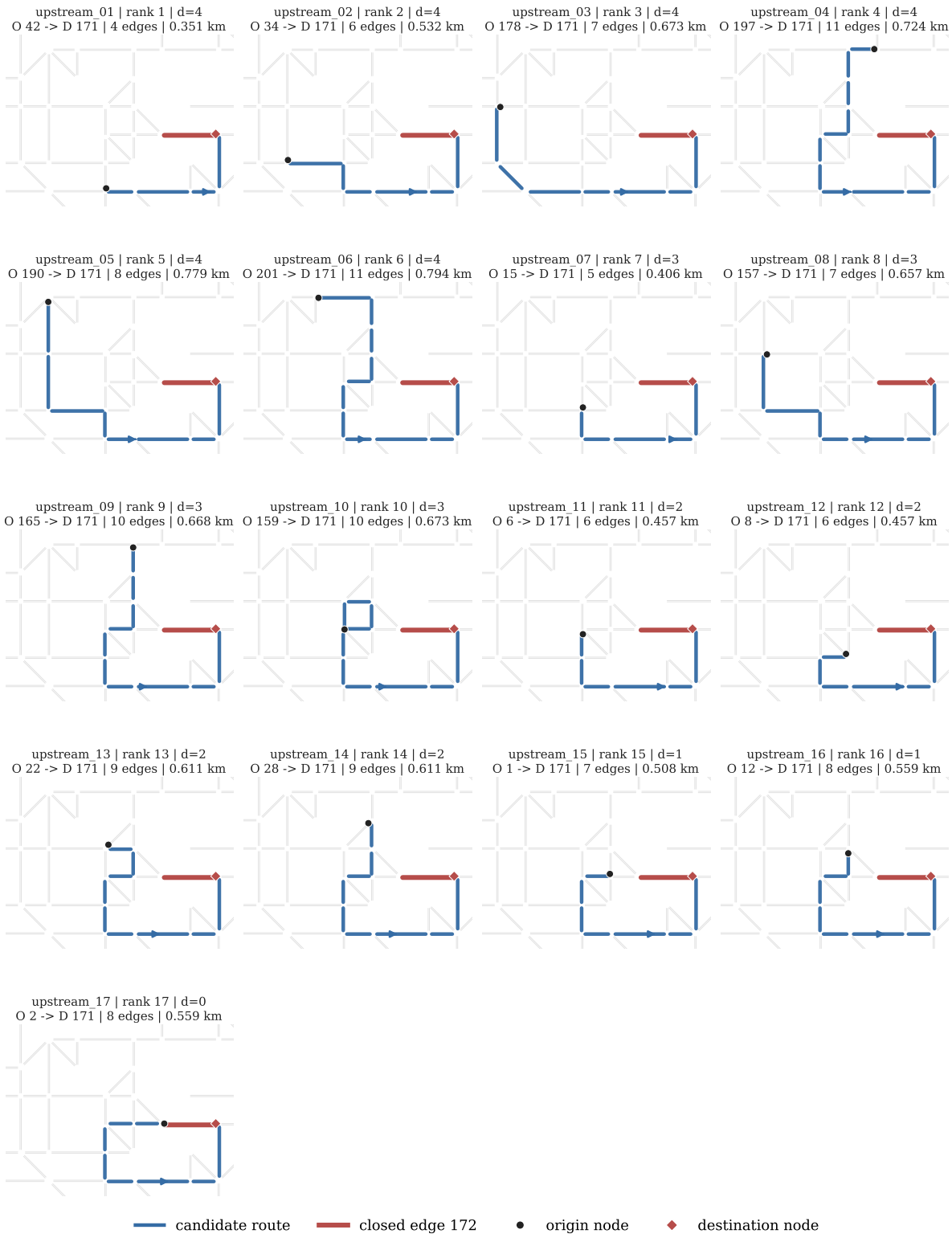


Figure 19: The 17 candidate deviation plans produced by the upstream BFS method for the closure of edge 172. Each panel isolates one alternative and lists its origin node, destination node, BFS depth, edge count and detour length. All alternatives terminate at destination junction 171, while their origin junctions vary by upstream depth.

Compared with the Yen output, the upstream plans cover a much broader spatial

footprint. All Yen paths share the single node 2 origin. In contrast, the upstream set spans 17 distinct interception points before the closure. This is the main practical benefit of the automated approach in this experiment: the manual plans describe two expert-selected corridors from the closure entrance, whereas the upstream method also proposes where traffic can be intercepted earlier if queues begin to propagate. Plans are passed to the route rewriter sorted depth-descending so the most spatially specific plan is tried first, falling back to shallower ones for vehicles that have already passed the deeper diversion points, `upstream_17` (depth 0, junction 2) serves as the guaranteed fallback for all affected vehicles.

4.4.1 Evaluator verdict

The evaluator assessed the 17 upstream-generated plans on the same three criteria: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip and the flow-weighted congestion proxy ϕ_{net}. All upstream plans are feasible, meaning none reduces the number of completed trips below the baseline.

The Pareto screening keeps six efficient alternatives: `upstream_04`, `upstream_08`, `upstream_13`, `upstream_14`, `upstream_15` and `upstream_16`. Among them, `upstream_16` gives the best time-based performance, with the lowest mean incremental delay (3.22 vh) and the lowest mean added travel time (+0.058 min per completed trip). However, it has the highest congestion proxy on the Pareto front, with $\phi_{\text{net}} = 0.383$. Conversely, `upstream_15` has the lowest congestion proxy ($\phi_{\text{net}} = 0.380$), but a higher delay and travel-time penalty. This creates a genuine multi-criteria trade-off rather than a single plan dominating all others.

The remaining 11 upstream alternatives are Pareto-dominated and are removed before weighted scoring. With equal reference weights, the additive score ranks `upstream_16` first, followed by `upstream_14`, `upstream_04`, `upstream_08`, `upstream_15` and `upstream_13`. The robustness test supports this recommendation: drawing $N = 20{,}000$ weight vectors uniformly from the weight simplex (Dirichlet$(1, 1, 1)$ sampling), `upstream_16` ranks first in 68.48% of draws. The next most robust first-rank alternative is `upstream_15`, with 19.22%, while `upstream_04` and `upstream_08` rank first in about 6% of draws each. The evaluator therefore recommends `upstream_16` as the best upstream-generated deviation plan for this closure, while noting that the final choice remains sensitive to how strongly the congestion proxy is weighted.

4.5 Comparison across generation strategies

Plan	Δ delay (vh)	Δ travel (min)	ϕ_{net}
`plan_1`	6.36	0.090	0.392
`yen_01`	4.35	0.070	0.380
`upstream_16`	3.22	0.058	0.383

Table 6: Best-ranked plan from each generation strategy in the draft experiment. Delta delay is reported in vehicle-hours, delta travel in minutes per completed vehicle and ϕ_{net} as the flow-weighted congestion proxy used by the evaluator.

Across the three plan families, the upstream method produces the best time-based result in this draft experiment. The best manual plan (`plan_1`) increases delay by 6.36 vh, the best Yen plan (`yen_01`) by 4.35 vh and the best upstream plan (`upstream_16`) by 3.22 vh. Relative to the best manual plan, the best Yen plan reduces the incremental delay by about 32% and the best upstream plan by about 49%. This does not mean the expert-created plans are poor, rather, it illustrates why automated generation is worth adding to the workflow. It can keep the manually selected corridors in the candidate set while also surfacing shorter or earlier-interception alternatives that would be tedious to enumerate manually. The comparison remains preliminary because the plan families were not all evaluated with the same number of Monte Carlo replications.

4.6 Real-neighbourhood validation: Flagey

To verify that the workflow is not tied to the regular geometry of the toy network, the same manually-defined-plan protocol was repeated on a small road-only SUMO network around Flagey in Brussels. The network was extracted from OpenStreetMap and filtered to passenger-car roads before conversion to SUMO. It contains 342 directed non-internal road edges, 364 lanes, 201 non-internal junctions and 18.66 lane-kilometres. A simple 3×3 TAZ grid was generated over the largest private-vehicle strongly connected component, producing 9 traffic analysis zones. This zoning is intentionally coarse: it is sufficient for a reproducible pipeline validation, but it is not a calibrated Brussels travel-demand model. Figure 20 shows this extracted topology, the generated TAZ grid, the selected closure and the three candidate detours.

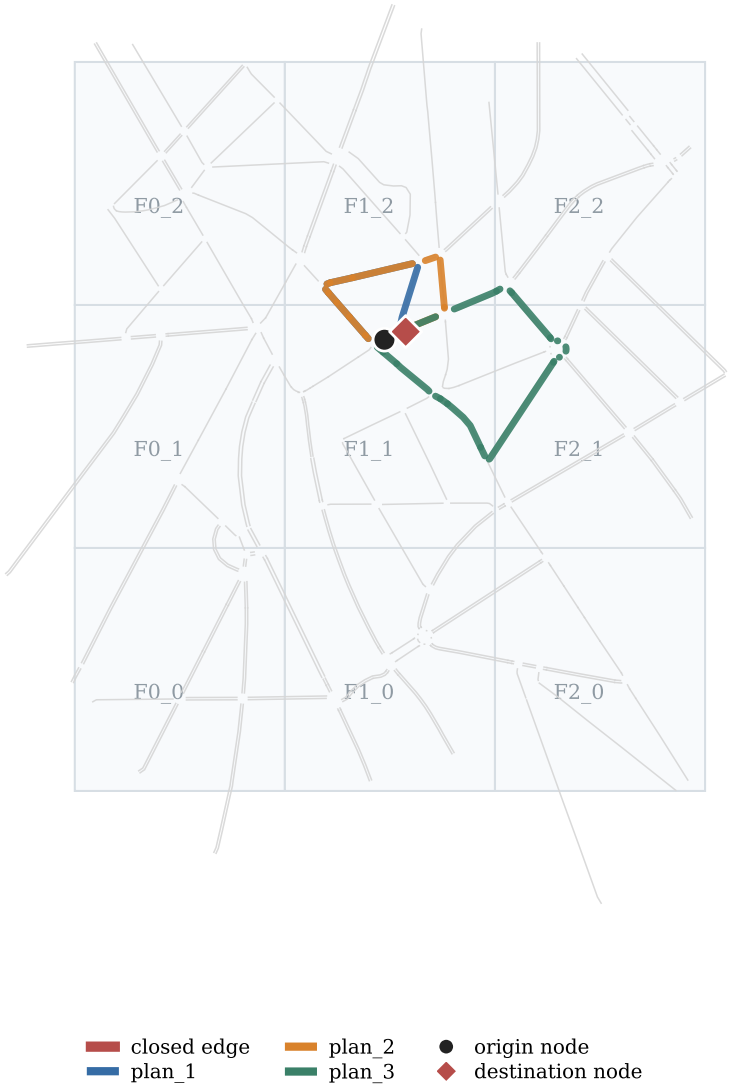


Figure 20: Flagey road network used for the real-neighbourhood validation. The grey graph is the road-only SUMO network, the light grid shows the 9 generated TAZs, red marks the closed edge and the three coloured paths show the candidate detours evaluated by Monte Carlo simulation. The common origin and destination are marked as O and D, with the exact node IDs reported in the legend.

The scenario generator produced 2,400 trip requests over one synthetic day. After stochastic routing with `duarouter`, 2,371 vehicles were present and completed in every baseline and plan simulation, the same completed count across all alternatives means the feasibility comparison is not affected by different vehicle losses. The temporal demand profile keeps the same two-peak structure as the toy experiment, with a morning peak

around 08:00 and the largest hourly volume at 17:00. Figure 21 shows the resulting hourly demand profile for this validation scenario.

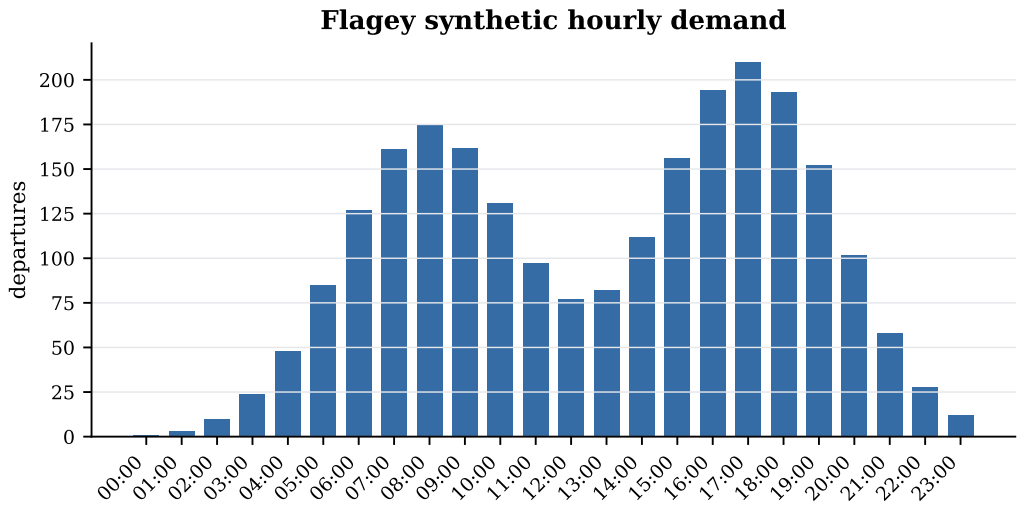


Figure 21: Synthetic hourly demand for the Flagey validation scenario. The profile is generated by the same scenario-generator model as the toy experiment, scaled down to 2,400 daily trip requests for a compact neighbourhood simulation.

The closed road segment is SUMO edge `-414404581#0`. It was selected because it lies in the central part of the extracted neighbourhood and admits several topology-valid source-to-target alternatives once the edge itself is blocked. Three candidate detours were generated from the same immediate closure endpoints: origin node cluster_207124856_3121275054_4240336304_5121849273_#2more and destination node cluster_26085294_3121275079_3121275084_3121275100_#1more. They were evaluated over 30 Monte Carlo runs with the same randomised-routing protocol used above. The three alternatives reroute the same average number of vehicles, 233.8 vehicles per run, corresponding to 9.86% of the completed daily demand. The comparison therefore isolates the quality of the replacement path rather than differences in how many vehicles are affected.

Plan	Edges	Length (km)	Delay	Δ delay	Δ travel
Baseline	–	–	27.73	–	–
`plan_1`	9	0.282	29.07	1.34	0.091
`plan_2`	12	0.322	28.76	1.03	0.097
`plan_3`	18	0.546	30.49	2.76	0.203

Table 7: Main Flagey Monte Carlo aggregates over 30 runs. Delay and delta delay are in vehicle-hours, delta travel is in minutes per completed vehicle. Each candidate reroutes the same affected population, 233.8 vehicles per run on average.

4.6.1 Evaluator verdict

The Flagey result is deliberately more nuanced than the two-plan manual result on *city_tiny*. `plan_3` is clearly inferior: it has the largest incremental delay (+2.76 vh),

the largest mean travel-time penalty (+0.203 min per completed trip) and the largest congestion proxy ($\phi_{\text{net}} = 0.338$). It is therefore Pareto-dominated by both `plan_1` and `plan_2`.

The choice between `plan_1` and `plan_2` is a genuine multi-criteria trade-off. `plan_2` produces the lowest mean incremental delay, +1.03 vh compared with +1.34 vh for `plan_1`. However, `plan_1` has the lower mean travel-time penalty (+0.091 min compared with +0.097 min) and the lower congestion proxy ($\phi_{\text{net}} = 0.309$ compared with 0.327). With equal reference weights across delay, travel time and the congestion proxy, the additive MCDA score therefore ranks `plan_1` first. The robustness test supports this recommendation but not unanimously: under 20,000 uniformly sampled weight vectors on the three-criterion simplex, `plan_1` ranks first in 74.51% of draws and `plan_2` in 25.49% of draws. Figure 22 summarises this trade-off and the final evaluator ranking.

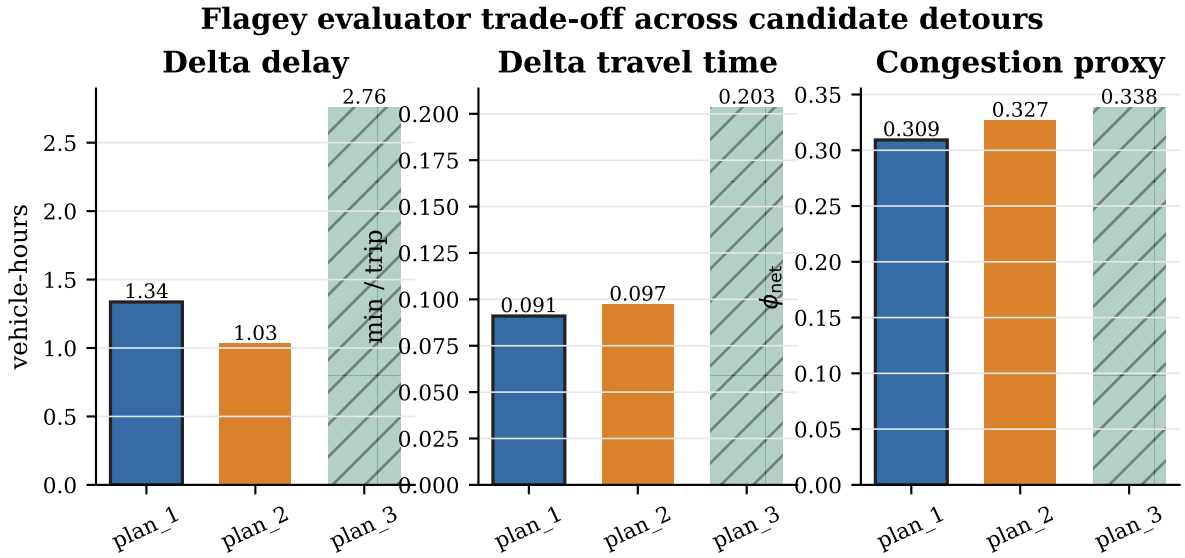


Figure 22: Evaluator trade-off for the Flagey validation experiment. The hatched bars indicate the Pareto-dominated `plan_3`. The selected plan, `plan_1`, is not the minimum-delay alternative, but it gives the best combined result once travel-time and congestion-proxy criteria are included. All three alternatives use the same origin and destination nodes shown in Figure 20.

This validation does not claim calibrated traffic performance for the Flagey area. The demand is synthetic, the TAZ zoning is automatic and the closure is an illustrative central edge chosen to exercise the route-rewriting logic. What it does show is that the thesis pipeline remains operational when the regular toy grid is replaced by a real urban street topology: the scenario generator produces routable demand, the plan representation remains valid, Monte Carlo route variation is reproducible over 30 runs and the evaluator returns an interpretable recommendation with an explicit trade-off rather than a purely visual judgement.

5 Discussion

5.1 Usefulness of the framework

The main value of the work is that it turns deviation planning from a manually inspected routing problem into a reproducible evaluation workflow. Starting from a SUMO network, a TAZ layer and a small set of configuration parameters, the framework produces synthetic demand, generates candidate detours, rewrites affected vehicle routes, runs paired simulations and ranks the resulting plans with the same criteria. This is useful even when the absolute numerical results are not yet calibrated to a real city, because it provides a controlled environment in which different diversion strategies can be compared on equal terms.

The experiment also shows that automated candidate generation can add information beyond the manually selected detours. In the *city_tiny* case, the best Yen-generated plan reduces incremental delay relative to the best manual plan, while the upstream method finds earlier interception points that are not visible when the detour is defined only from the source of the closed edge. This does not replace expert judgement: a traffic manager still needs to check local constraints, signing possibilities and policy priorities. However, the automated methods widen the candidate set and make it easier to notice alternatives that would otherwise be tedious to enumerate by hand.

A second useful aspect is the paired Monte Carlo evaluation. Each candidate plan is compared with the matching baseline under the same demand and routing seed, so the reported deltas describe the effect of the plan rather than an uncontrolled difference between simulation runs. The stochastic repetitions also expose whether the recommendation is robust. For the manual plans, the ranking is unambiguous because `plan_1` dominates `plan_2`. For the Yen and upstream candidates, the results show a more realistic decision problem: the best time-based plan is not always the best plan on the congestion proxy, so the preferred option depends on how the decision maker weights the criteria.

This is an important implication of using a Pareto-based decision layer. When several candidates remain non-dominated, the evaluation should not be read as having failed to identify the best deviation plan. Rather, it reveals that the alternatives embody different compromises. One plan may minimise total network delay while increasing the burden on the vehicles that are directly affected by the closure; another may be less efficient on average but avoid severe spillback, distribute traffic more evenly across the network or perform more robustly across stochastic runs. The final choice is therefore partly a normative and operational decision. Besides mean delay and travel time, a traffic authority may need to evaluate the spatial distribution of congestion, the equity of the burden placed on different origin-destination pairs or neighbourhoods, the variance and worst-case outcomes across Monte Carlo runs, the sensitivity of the ranking to MCDA weights, and the practical feasibility of communicating and enforcing the plan on the ground. In such cases, the framework is most useful when it exposes the trade-off surface and identifies a set of defensible alternatives, rather than forcing all candidates into a single apparently objective ranking.

The decision layer is intentionally simple, but that simplicity is useful for operational interpretation. Feasibility filtering, Pareto screening and weight-sensitivity analysis make the recommendation traceable: a rejected plan can be rejected because it is infeasible, because another plan dominates it, or because it only wins under a narrow set of weights. The framework therefore acts less as an automatic prescription tool and more as a structured decision-support tool that documents why a deviation plan is preferred.

5.2 Limitations

These contributions remain bounded by several methodological and modelling choices. The following limitations are grouped by the stage of the pipeline they affect and are discussed in terms of the threat they pose to internal and external validity.

5.2.1 Single-closure assumption

The deviation plan maker and the route rewriter are both designed around a single closed edge at a time. In practice, roadworks often affect a contiguous segment spanning several edges or a whole junction and emergency closures occasionally create simultaneous disruptions in different parts of the network. The current framework does not support overlapping or cascading closures: running the pipeline twice for two independent edges produces plans that are optimal in isolation but may conflict spatially when applied together. Extending the blocked-edge set to a multi-edge closure is technically straightforward, but the interaction between concurrent detour plans (which detour plan causes what effects) needs to be studied.

5.2.2 No intra-zone demand

Demand is synthesised from TAZ-level OD matrices, which by construction assign all trip origins and destinations to zone centroids. Trips whose true origin and destination both lie inside the same TAZ are therefore excluded from the demand entirely. In the *city_tiny* scenario each TAZ is small and the synthetic demand is deliberately cross-zonal, so this simplification has limited impact, however, on real networks with large zones, intra-zonal demand can represent a significant share of daily trips. Vehicles that depart and arrive within the same zone are also the most likely to use short local roads in the vicinity of the closed edge, meaning that the framework may underestimate the fraction of demand directly affected by the closure.

5.2.3 Hourly OD time resolution

OD matrices are aggregated over one-hour intervals. Sub-hourly demand surges (such as the sharp traffic spike at the end of a sports event or the fifteen-minute burst following a school bell) are smoothed away. Because duarouter distributes departures uniformly within each hourly bucket, the simulated traffic loading during the closure is an average over the full hour rather than a realistic second-by-second profile. This can cause the MC evaluation to underestimate peak-period congestion on detour corridors, which in turn

biases the incremental delay estimates downward for high-density time windows.

5.2.4 Synthetic network only

All presented results come from the *city_tiny* procedurally generated network. Because the network topology, TAZ layout and demand profile are artificial, the absolute numbers reported (vehicle-hours of delay, phi_net values) cannot be interpreted as predictions for any real city. The framework's correctness is demonstrated on a controlled, reproducible test case, but generalization to real urban networks requires validation against observed data from an actual closure event.

5.2.5 Demand fixed across Monte Carlo runs

Stochasticity in the Monte Carlo loop enters exclusively through duarouter's randomised edge weights. The vehicle population (departure times, origin-destination pairs) is generated once and held constant across all 30 runs. Real-world demand is itself uncertain: travellers receive information about the closure at different times, some cancel their trips and others shift their departure time. Demand-side uncertainty is therefore not captured and the variance in the MC outputs reflects only route-assignment variability, not the full uncertainty of real closure impacts.

5.2.6 Single vehicle class

The simulation models only private passenger cars. Freight vehicles, buses, trams, cyclists and pedestrians are absent from the demand. This matters particularly for plan evaluation: heavy goods vehicles have different routing constraints (height and weight restrictions, turning radii) that may make some candidate detour edges physically infeasible for them even if they are optimal for private cars. Emergency services are similarly excluded, so the impact of a closure on response times is not assessed.

5.2.7 Static plan generation

Plans are computed once from the network topology at the moment the closure is declared. The framework does not adapt plans to the real-time traffic state: if a candidate detour is already near saturation before the closure occurs, the plan generator has no mechanism to avoid it. A natural extension would integrate a live traffic data feed so that edge weights at plan generation time reflect current conditions rather than free-flow speeds.

6 Conclusion

Disclosure of AI Tools

AI-assisted tools were used for language, formatting, LaTeX hygiene and software-development support during the preparation of this work. The author remains responsible for the technical choices, analysis, results and conclusions.